Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

exdqlm: An R Package for Estimation and Analysis of Flexible Dynamic Quantile Linear Models

Antonio Aguirre
University of California Santa Cruz

Raquel Barata
University of California Santa Cruz

Raquel Prado
University of California Santa Cruz

Bruno Sansó
University of California Santa Cruz

Abstract

We present the R package **exdqlm** for Bayesian quantile regression, with primary emphasis on dynamic state-space quantile models for time series. The package is built around flexible dynamic quantile linear models (exDQLMs), which use the extended asymmetric Laplace (exAL) family, a parametric extension of the asymmetric Laplace (AL) commonly used in quantile regression. The software provides posterior simulation via Markov chain Monte Carlo (MCMC) and fast approximate posterior inference via variational Bayes (VB), supporting posterior uncertainty quantification while also providing a computationally efficient option for longer time series. It also supports static exAL quantile regression with regularized priors, transfer-function models for nonlinear input effects at a given quantile, post hoc posterior-predictive synthesis across separately fitted quantiles, forecasting, and several quantitative and visual diagnostics for model evaluation.

Keywords: dynamic quantile regression, dynamic models, asymmetric Laplace, variational Bayes, coordinate ascent variational inference, quantile regression.

1. Introduction

Quantile regression has become a widely used alternative to mean-based modeling in static settings. However, dynamic quantile methods and accompanying software remain comparatively limited. In Bayesian parametric quantile regression, the asymmetric Laplace (AL) likelihood has played a central role because it is linked to the check-loss function used in classical quantile regression (Yu and Moyeed 2001; Koenker 2005) and admits convenient mixture representations for posterior computation. These computational advantages have

made AL-based models attractive, even though the AL can be restrictive as an error model. More flexible error distributions have been studied extensively in the Bayesian nonparametric literature (Kottas and Krnjajić 2009; Reich, Bondell, and Wang 2009; Taddy and Kottas 2010), however the resulting methods can be computationally demanding for dynamic models and longer time series. Yan, Zheng, and Kottas (2025) introduce the extended asymmetric Laplace (exAL); a parametric extension of the AL that preserves its computational advantages while relaxing some of its restrictive features. Barata, Prado, and Sansó (2022) use this family to develop a flexible dynamic quantile linear model (exDQLM), which is the model implemented in our R package **exdqlm**.

The current R software ecosystem for quantile regression is broad, but the capabilities most relevant to this paper are distributed across software with different aims. Table 1 gives a compact roadmap of representative packages using the distinctions most important for positioning **exdqlm**: modeling scope, temporal or latent-state structure, inference strategy, and handling of multiple quantile levels. The table is selective rather than exhaustive; its purpose is to identify the software gap addressed by a dedicated Bayesian dynamic quantile state-space workflow.

The package **exdqlm** provides a unified workflow for Bayesian inference, forecasting, and model assessment for quantile models fitted one quantile at a time. Its core functionality is dynamic state-space quantile modeling, supported by posterior simulation via MCMC and a faster variational Bayes (VB) approximation. The package also includes transfer-function exDQLMs for nonlinear input effects at a given quantile, static exAL quantile regression with shrinkage priors, and a synthesis step that combines quantile-specific posterior predictive draws from separately fitted quantile models into a unified posterior predictive distribution. Forecasting and a range of visual and quantitative diagnostics are available throughout. The package is released under the MIT License and is available from the Comprehensive R Archive Network (CRAN) at `https://CRAN.R-project.org/package=exdqlm`.

The frequentist package **quantreg** (Koenker 2025) remains the broad general-purpose baseline, covering linear, nonlinear, nonparametric, censored, and dynamic quantile regression, together with bootstrap-based and related uncertainty summaries. Its `dynrq()` function supports dynamic linear quantile regression through time-series regression formulas involving lags, differences, trends, seasonal factors, and harmonic terms, while preserving time-series attributes. However, the dynamic structure enters through the design matrix rather than through latent states: once these regressors are constructed, estimation proceeds through the standard **quantreg** fitting routines for a fixed coefficient vector. In that sense, `dynrq()` is a regression-style time-series interface rather than a latent-state state-space model, so it does not provide filtered or smoothed state summaries.

Most other R packages relevant here implement static quantile regression or target a different multi-quantile objective. Among the static Bayesian options, **bayesQR** (Benoit, Al-Hamzawi, Yu, and Van den Poel 2023) provides Bayesian quantile regression under the AL working likelihood with an adaptive-lasso option, whereas **pqrBayes** (Fan, Wu, Ren, Li, and Zhou 2026) emphasizes penalized Bayesian quantile regression with spike-and-slab and horseshoe-family shrinkage priors. The broader Bayesian modeling package **brms** (Bürkner 2026) is also relevant because it supports AL-based quantile regression within a flexible multilevel, nonlinear, and additive regression framework, and can incorporate regression-level autocorrelation terms. However, it is not organized as a dedicated quantile state-space workflow, and its documentation does not present filtered or smoothed state summaries of the type targeted by

Package	Main scope	Time-series or state-space role	Inference or computation	Multiple-quantile handling
quantreg	General frequentist quantile regression	Formula-based time-series regressors via `dynrq()`; no latent states	Optimization-based fitting; frequentist and bootstrap summaries	Vector-valued quantile levels; post hoc rearrangement
bayesQR	Bayesian AL quantile regression	Static regression; no temporal or state-space component	MCMC under the AL likelihood; optional adaptive lasso	Vector-valued quantile levels; output organized by fitted quantile
pqrBayes	Bayesian penalized quantile regression	Static or varying-coefficient regression; no state-space component	MCMC with spike-and-slab, Laplace, or horseshoe-family priors	One target quantile per fit
brms	General Bayesian regression with an AL family	Autocorrelation structures available; no dedicated quantile state-space workflow	Stan-based posterior inference	Target quantile set through AL parameter
qgam	Smooth additive quantile regression	Static additive smooths; no state-space component	Automatic smoothing and learning-rate calibration; PIRLS or Newton fitting	Single-quantile `qgam()`; multi-quantile `mqgam()`
qrjoint	Bayesian joint linear quantile regression	Static regression over a predictor domain; no state-space component	Adaptive MCMC for quantile-level coefficient functions	Joint noncrossing quantile planes
dlm	Gaussian dynamic linear models	Latent Gaussian state-space models	Maximum likelihood, Kalman filtering/smoothing, and Bayesian analysis	Not a quantile-regression package
exdqlm	Bayesian exAL quantile regression	Latent quantile state-space models, forecasting, and transfer-function effects	MCMC and variational Bayes; static shrinkage-prior options	Posterior-predictive synthesis from separately fitted quantiles

Table 1: Representative R packages relevant to quantile and state-space modeling. Entries summarize documented primary interfaces along the dimensions used to position **exdqlm**; they are selective rather than exhaustive. AL denotes asymmetric Laplace, exAL its extended version, PIRLS penalized iteratively reweighted least squares, and VB variational Bayes.

exdqlm. Packages such as **qgam** (Fasiolo and Griffiths 2025), **qrjoint** (Tokdar and Cunningham 2025), **quantregGrowth** (Muggeo 2025), and **qrcm** (Frumento 2025) address additive, joint, non-crossing, or coefficient-modeling formulations. Other packages focus on specialized static settings, including penalized frequentist fitting across quantiles in **rqPen** (Sherwood, Li, and Maidman 2026), convolution-smoothed linear quantile regression with efficient gradient-based fitting in **conquer** (He, Pan, Tan, and Zhou 2023), and robust or censored-response modeling in **lqr** (Galarza, Benites, Bourguignon, and Lachos 2024).

Related implementations also exist outside R. In Python, **statsmodels** (statsmodels developers 2025) provides linear quantile regression via iterative reweighted least squares. Julia offers **QuantReg.jl** (Fogarty 2020), and MATLAB includes tools such as `fitrqlinear` and `fitrqnet` (MathWorks 2026). These implementations are useful reference points for the broader software ecosystem, but they are better viewed as general or static quantile-regression tools

than as direct counterparts to a dedicated package for Bayesian dynamic quantile state-space modeling.

Viewed against the comparison in Table 1, **exdqlm** is not intended to replace broad general-purpose quantile-regression toolkits, additive quantile-regression packages, or software designed specifically for joint multi-quantile estimation. Rather, it addresses a more specific need: a unified Bayesian workflow for dynamic/state-space quantile modeling that combines posterior simulation with a fast variational approximation. Our package **exdqlm** fills this need while also using the more flexible exAL likelihood, supporting static exAL regression with shrinkage priors ranging from ridge to regularized horseshoe variants, and providing transfer-function models, forecasting, diagnostics, and post hoc posterior-predictive synthesis from separately fitted quantile models. General state-space packages such as **dlm** (Petris and Gilks 2018) and **dynr** (Ou, Hunter, Chow, Ji, Chen, Hung, Lee, Li, and Park 2021) remain useful for constructing dynamic components, but they do not provide dedicated quantile-regression workflows.

The remainder of the paper is organized as follows. In Section 2, we describe the modeling framework, including the algorithms used for posterior inference and our methods for including nonlinear relationships. We also discuss static quantile regression as a special case of the framework. In Section 3 we detail the model diagnostics and forecasting method implemented in the package. Then, in Section 4, we walk through four step-by-step examples on how to use the package. Finally, in Section 5, we present a summary of the package capabilities.

2. Extended dynamic quantile linear models

Consider a set of scalar time-evolving responses, y_t, for times $t = 1, \ldots, T$. For each t, an extended dynamic quantile linear model (exDQLM) for inference on a single p_0-th quantile is defined by

$$\text{Observation equation:} \quad y_t = \mathbf{F}_t'\boldsymbol{\theta}_t + \epsilon_t, \qquad \epsilon_t \sim \text{exAL}_{p_0}(0, \sigma, \gamma) \tag{1}$$

$$\text{System equation:} \quad \boldsymbol{\theta}_t = \mathbf{G}_t\boldsymbol{\theta}_{t-1} + \boldsymbol{\omega}_t, \qquad \boldsymbol{\omega}_t \sim \text{N}(\mathbf{0}, \mathbf{W}_t). \tag{2}$$

Here $\mathbf{F}_t$ is the $q \times 1$ regression vector of the covariates corresponding to the $q \times 1$ parameter vector $\boldsymbol{\theta}_t$ at time t, $\mathbf{G}_t$ is the $q \times q$-dimensional evolution matrix defining the structure of the parameter vector evolution in time, and the normal distribution according to which the system error $\boldsymbol{\omega}_t$ evolves is q-variate. The observational errors ϵ_t of the exDQLM follow an exAL distribution for fixed quantile p_0, whose density is denoted as $\text{exAL}_{p_0}(\cdot)$, and it is such that $\int_{-\infty}^{0} \text{exAL}_{p_0}(\epsilon_t|0, \sigma, \gamma)\text{d}\epsilon_t = p_0$. Thus, the observation equation in Equation (1) implies that $\mathbf{F}_t'\boldsymbol{\theta}_t$ corresponds to the p_0-quantile of y_t, even if this is not made explicit in the notation above. Table 2 summarizes how the state-space quantities and prior hyperparameters introduced in this section are represented in the **exdqlm** function inputs.

The exAL distribution, introduced by Yan *et al.* (2025), extends the location-scale mixture representation of the AL (Kozumi and Kobayashi 2011). For fixed p_0, the skewness parameter γ is restricted to an interval (L, U), where L and U are the negative and positive roots of $g(\gamma) = 1 - p_0$ and $g(\gamma) = p_0$, respectively, with $g(\gamma) = 2\Phi(-|\gamma|)\exp(\gamma^2/2)$. This yields the following mixture representation of $\text{exAL}_{p_0}(0, \sigma, \gamma)$:

$$\int\int_{\mathbb{R}^+\times\mathbb{R}^+} \text{N}(y_t|C(p,\gamma)\sigma|\gamma|s_t + A(p)v_t, \sigma B(p)v_t)\text{Exp}(v_t|\sigma)\text{N}^+(s_t|0,1)\text{d}v_t\text{d}s_t. \tag{3}$$

Here, $p = p(p_0, \gamma) = I(\gamma < 0) + \{[p_0 - I(\gamma < 0)]/g(\gamma)\}$ where $g(\gamma) = 2\Phi(-|\gamma|)\exp(\gamma^2/2)$, and $\Phi(\cdot)$ denotes the standard normal CDF. $A(p)$, $B(p)$, $C(p, \gamma)$ are the functions of p and γ: $A(p) = \frac{1-2p}{p(1-p)}$, $B(p) = \frac{2}{p(1-p)}$, $C(p, \gamma) = [I(\gamma > 0) - p]^{-1}$. Lastly, $\text{Exp}(v|\sigma)$ denotes the exponential distribution with mean σ, and $\text{N}^+(s_t|0, 1)$ denotes a normal distribution truncated to the positive reals with mean 0 and variance 1. This mixture representation of the exAL can be exploited to rewrite the exDQLM as the following hierarchical model for $t = 1, \ldots, T$:

$$y_t|\boldsymbol{\theta}_t, \sigma, \gamma, v_t, s_t \sim \text{N}(y_t|\mathbf{F}_t'\boldsymbol{\theta}_t + C(p, \gamma)\sigma|\gamma|s_t + A(p)v_t, \sigma B(p)v_t) \tag{4}$$

$$v_t, s_t|\sigma \sim \text{Exp}(v_t|\sigma)N^+(s_t|0, 1) \tag{5}$$

$$\boldsymbol{\theta}_t|\boldsymbol{\theta}_{t-1}, \mathbf{W}_t \sim \text{N}(\mathbf{G}_t\boldsymbol{\theta}_{t-1}, \mathbf{W}_t). \tag{6}$$

This hierarchical form is leveraged for the posterior inference implemented within the **exdqlm** package.

2.1. Prior specification

Under a Bayesian formulation and given p_0, we specify priors for the initial state vector $\boldsymbol{\theta}_0$, scale parameter σ and skewness parameter γ. For $\boldsymbol{\theta}_0$, we assume a q-variate conjugate normal prior $\boldsymbol{\theta}_0 \sim \text{N}(\mathbf{m}_0, \mathbf{C}_0)$. For σ, we assume a conjugate inverse-gamma prior denoted as $\text{IG}(a_\sigma, b_\sigma)$. In many cases a strong prior on σ is necessary to guarantee fast posterior convergence and estimation. This is illustrated in the examples of Section 4. The parameter γ has bounded support over the interval (L, U) where L is the negative root of $g(\gamma) = 1 - p_0$ and U is the positive root of $g(\gamma) = p_0$ (Yan *et al.* 2025). In contrast to the flat prior suggested by Yan *et al.* (2025), we find that using a proper prior on the skewness parameter promotes reliable posterior inference. Thus, we place a Student-t distribution truncated to the interval (L, U) as the prior for γ, i.e. $\gamma \sim \text{t}_{(L,U)}(m_\gamma, s_\gamma)$ with ν_γ degrees of freedom. Table 2 gives the corresponding **exdqlm** input names and default values when applicable.

R input	`model`				`PriorSigma`		`PriorGamma`		
Mathematical symbol	$\mathbf{F}_{1:T}$	$\mathbf{G}_{1:T}$	$\mathbf{m}_0$	$\mathbf{C}_0$	a_σ	b_σ	m_γ	s_γ	ν_γ
R component	`FF`	`GG`	`m0`	`C0`	`a_sig`	`b_sig`	`m_gam`	`s_gam`	`df_gam`
Default if omitted	–	–	$\mathbf{0}_q$	$10^3\mathbf{I}_q$	2.1	1.1	0	1	1

Table 2: Translation from exDQLM notation to **exdqlm** function inputs. The `model` object stores state-space components and the initial-state prior; `PriorSigma` and `PriorGamma` store hyperparameters for σ and γ. A dash indicates no generic default because `FF` and `GG` are determined by the chosen component or supplied model. Here $\mathbf{0}_q$ and $\mathbf{I}_q$ denote a q-vector of zeros and the q-dimensional identity matrix.

2.2. Posterior estimation

The hierarchical representation of the exDQLM facilitates posterior simulation using Markov chain Monte Carlo (MCMC) algorithms. This is implemented in the function `exdqlmMCMC()`. Gibbs sampling steps are used to update the sets of latent variables s_t and v_t for $t = 1, \ldots, T$. The dynamic regression coefficients are sampled using a forward-filtering backward-sampling algorithm (Carter and Kohn 1994; Frühwirth-Schnatter 1994). The scale and skewness parameters are updated separately from the latent-state blocks. By default, the package uses a

slice-sampling update for the (σ, γ) block, while joint random-walk Metropolis-Hastings (MH) alternatives are also available. When desired, the chain can be warm-started from the variational approximation; this initialization is separate from the subsequent MCMC proposal kernel. An example of this functionality can be found in Section 4.1. We perform a total of $N_{BURN} + N_{MCMC}$ MCMC iterations, discarding the first N_{BURN} and retaining the remaining N_{MCMC} posterior samples. The object created by `exdqlmMCMC()` includes posterior samples of all parameters $(\boldsymbol{\theta}_{1:T}, v_{1:T}, s_{1:T}, \gamma, \sigma)$, filtered and smoothed summaries for the state vector $\boldsymbol{\theta}_{1:T}$, and diagnostics for the proposal kernel used in the (σ, γ) update.

For purposes of model selection or large time series data sets, posterior sampling with the MCMC algorithm can be computationally demanding. To address this, the package includes a variational Bayes (VB) routine for fast approximate posterior estimation. The default routine, `exdqlmLDVB()`, uses a mean-field approximation together with a Laplace–delta treatment of the nonconjugate (σ, γ) block. The variational Bayes (VB) framework approximates the posterior distribution with partitioned variational distributions (Ostwald, Kirilina, Starke, and Blankenburg 2014; Beal 2003). Under a mean-field factorization, this induces a blockwise coordinate-ascent variational inference scheme. For the exDQLM, the blocks associated with the dynamic coefficients and latent variables admit recognizable closed-form updates, while the joint variational distribution of σ and γ is nonconjugate. Following the nonconjugate VB strategy of Wang and Blei (2013), we approximate this block on transformed coordinates and use a second-order delta method to evaluate the expectations of functions of (σ, γ) needed by the remaining variational updates. The dynamic regression coefficients are updated within each iteration using a forward-filtering backward-smoothing algorithm. Convergence is assessed jointly through the state, scale, skewness, and ELBO diagnostics. After the algorithm reaches convergence with tolerance ϵ_{tol}, N_{SAMP} samples are drawn from the variational distributions, resulting in an approximate sample of the posterior distribution. The output of `exdqlmLDVB()` includes the approximate posterior samples as well as the parameters of the variational distributions for all parameters $(\boldsymbol{\theta}_{1:T}, v_{1:T}, s_{1:T}, \gamma, \sigma)$. For the (γ, σ) block, the return also contains the transformed mode, approximate covariance matrix, derived moments, and ELBO terms associated with the Laplace–delta approximation. Selected state-space computations in the dynamic routines are implemented in C++, with corresponding R code retained for parity checks and fallback use. Global package-level runtime options for backend routing and ELBO monitoring are summarized in Appendix Table 10. These options are distinct from the function-specific inference inputs listed in Table 3.

Table 3 shows selected core inference controls for the fitting functions `exdqlmMCMC()` and `exdqlmLDVB()`, their mathematical symbols, and their default values. The MCMC routine also accepts `Sig.mh` as an optional proposal covariance when `mh.proposal` selects a joint random-walk MH kernel; it is not used under the default slice-sampling update.

Control	`exdqlmMCMC()`			`exdqlmLDVB()`	
Mathematical symbol	$\mathcal{K}_{\sigma,\gamma}$	N_{BURN}	N_{MCMC}	ϵ_{tol}	N_{SAMP}
R argument	`mh.proposal`	`n.burn`	`n.mcmc`	`tol`	`n.samp`
Default if omitted	`"slice"`	2000	1500	0.1	200

Table 3: Selected core inference controls for `exdqlmMCMC()` and `exdqlmLDVB()`. Advanced control-list arguments, optional random-walk MH tuning inputs, and package-level backend options are documented separately.

2.3. Specification of the evolution covariance via discount factors

Both estimation algorithms, `exdqlmMCMC()` and `exdqlmLDVB()`, utilize discount factors to specify the time-evolving covariance matrices $\mathbf{W}_t$ introduced in Equation (2) (West, Harrison, and Migon 1985). More precisely, we define the evolution variance matrix $\mathbf{W}_t = \frac{1-\delta}{\delta}\mathbf{G}_t\mathbf{C}_{t-1}\mathbf{G}_t'$ where $\mathbf{C}_{t-1}$ denotes the posterior variance of the state vector $\boldsymbol{\theta}_{t-1}$ at time $t-1$ and δ is a discount factor such that $0 < \delta \leq 1$. The discount factor δ can be specified by the user via function parameter `df`. Selection of discount factors is typically done by optimizing a model-checking criterion. In practice we recommend predictive criteria such as the CRPS or PPLC discussed in Section 3, with the KL divergence as a complementary calibration diagnostic. A brief example of this method for discount factor selection can be found in Section 4.2.

Discounting by component of the state-space model is also a functionality of our estimation algorithms. Suppose the state vector is comprised of h components $\boldsymbol{\theta}_{it}$ of dimension q_i for $i = 1, \ldots, h$ such that $\boldsymbol{\theta}_t' = (\boldsymbol{\theta}_{1t}', \ldots, \boldsymbol{\theta}_{ht})'$ and $\sum_{i=1}^{h} q_i = q$. If $\mathbf{W}_{it}$ denotes the i^{th} component of the evolution covariance matrix such that $\mathbf{W}_t = \text{blockdiag}\{\mathbf{W}_{1t}, \ldots, \mathbf{W}_{ht}\}$ and δ_i denotes a discount factor for the i^{th} component, we can define the evolution variance matrix by component as $\mathbf{W}_{it} = \frac{1-\delta_i}{\delta_i}\mathbf{G}_{it}\mathbf{C}_{i,t-1}\mathbf{G}_{it}'$. Here $\mathbf{G}_{it}$ and $\mathbf{C}_{i,t-1}$ denote the i^{th} components of $\mathbf{G}_t$ and $\mathbf{C}_{t-1}$, respectively. The function parameters `df` and `dim.df` allow users to specify a set of component discount factors $(\delta_1, \ldots, \delta_h)$ and their dimensions $(q_1, \ldots, q_h)$, respectively, within the state-space model. Examples of this feature can be found in Section 4. For a full review of discount factors including discounting by component, see West *et al.* (1985).

2.4. Transfer function model

Quantifying the relationship between an input and a quantile of a response is a non-trivial task, particularly when the relationship is nonlinear. In mean-centric dynamic modeling, transfer functions are a straightforward method to incorporate variables which measure the combined effect of current and past regression effects (West *et al.* 1985). Within **exdqlm**, this extension is handled by augmenting the dynamic state-space representation conditional on a fixed transfer-function rate λ, and the resulting workflow is available through both VB and MCMC fitting routines.

For $t = 1, \ldots, T$ and transfer input vector $\mathbf{x}_t \in \mathbb{R}^k$, a transfer-function exDQLM with exponential decay is defined by

$$y_t|\boldsymbol{\theta}_t, \zeta_t, \gamma, \sigma \sim \text{exAL}_{p_0}(\mathbf{F}_t'\boldsymbol{\theta}_t + \zeta_t, \sigma, \gamma) \tag{7}$$

$$\boldsymbol{\theta}_t|\boldsymbol{\theta}_{t-1}, \mathbf{W}_t^{\theta} \sim \text{N}(\mathbf{G}_t\boldsymbol{\theta}_{t-1}, \mathbf{W}_t^{\theta}) \tag{8}$$

$$\zeta_t|\zeta_{t-1}, \boldsymbol{\psi}_{t-1}, \omega_t \sim \text{N}(\lambda\zeta_{t-1} + \mathbf{x}_t'\boldsymbol{\psi}_{t-1}, \omega_t) \tag{9}$$

$$\boldsymbol{\psi}_t|\boldsymbol{\psi}_{t-1}, \mathbf{W}_t^{\psi} \sim \text{N}(\boldsymbol{\psi}_{t-1}, \mathbf{W}_t^{\psi}). \tag{10}$$

Here $\zeta_t \in \mathbb{R}$ denotes the accumulated transfer effect on the quantile at time t, while $\boldsymbol{\psi}_t \in \mathbb{R}^k$ is the vector of instantaneous transfer coefficients associated with the components of $\mathbf{x}_t$. The mean contribution of the transfer block is therefore $\lambda\zeta_{t-1} + \mathbf{x}_t'\boldsymbol{\psi}_{t-1}$: the inner product $\mathbf{x}_t'\boldsymbol{\psi}_{t-1}$ captures the contemporaneous effect of the current inputs, and the parameter $\lambda \in (0, 1)$ controls how much of the previous accumulated effect is propagated forward. For $0 < \lambda < 1$, this propagated contribution decays geometrically. More explicitly, the contribution of $\mathbf{x}_t$ to

the quantile at time $t+h$ is $\lambda^h \mathbf{x}_t' \boldsymbol{\psi}_{t-1}$. Therefore, for each t and tolerance $\epsilon > 0$, any

$$k_t \geq \frac{\log(\epsilon) - \log(|\mathbf{x}_t' \boldsymbol{\psi}_{t-1}|)}{\log(\lambda)}$$

provides a lower bound on the number of time steps until the magnitude of the propagated effect falls below ϵ. The package reports the median of this bound over time for $\epsilon = 1\text{e}{-3}$.

Conditional on a fixed λ, say $\tilde{\lambda}$, we augment the dynamic fitting routines to incorporate the transfer-function structure by replacing $\mathbf{F}_t$, $\boldsymbol{\theta}_t$, $\mathbf{G}_t$, and $\mathbf{W}_t^{\theta}$ in Equations (1)-(2) with

$$\tilde{\mathbf{F}}_t' = (\mathbf{F}_t', 1, \mathbf{0}_k'), \qquad \tilde{\boldsymbol{\theta}}_t' = (\boldsymbol{\theta}_t', \zeta_t, \boldsymbol{\psi}_t'),$$

$$\tilde{\mathbf{G}}_t = \text{blockdiag}\{\mathbf{G}_t, \mathbf{G}_t^{tf}\}, \qquad \mathbf{G}_t^{tf} = \begin{pmatrix} \tilde{\lambda} & \mathbf{x}_t' \\ \mathbf{0}_k & \mathbf{I}_k \end{pmatrix},$$

and

$$\tilde{\mathbf{W}}_t = \text{blockdiag}\{\mathbf{W}_t^{\theta}, \omega_t, \mathbf{W}_t^{\psi}\}.$$

This augmentation, which extends the routines found in `exdqlmLDVB()` and `exdqlmMCMC()`, is implemented in the functions `exdqlmTransferLDVB()` and `exdqlmTransferMCMC()`.

Similar to the specification of $\mathbf{W}_t^{\theta}$ discussed in Section 2.3, discount factors are used to specify the transfer block variances ω_t and $\mathbf{W}_t^{\psi}$. In the package, `tf.df` can be supplied as a single shared discount factor, as a pair $(\delta_\zeta, \boldsymbol{\delta}_{\boldsymbol{\psi}})$ with a shared value for the whole $\boldsymbol{\psi}_t$ block, or componentwise across $(\zeta_t, \psi_{1,t}, \ldots, \psi_{k,t})$. A conjugate normal prior on $(\zeta_0, \boldsymbol{\psi}_0')' \sim N(\mathbf{m}_0^{tf}, \mathbf{C}_0^{tf})$, with $\mathbf{m}_0^{tf} \in \mathbb{R}^{k+1}$ and $\mathbf{C}_0^{tf} \in \mathbb{R}^{(k+1)\times(k+1)}$, completes the transfer-function model extension. Table 4 shows the additional transfer-function inputs, their mathematical symbols, and their accepted forms or defaults in the package interface.

Quantity	Mathematical symbol	R argument	Input/default
Transfer inputs	$\mathbf{x}_t$	`X`	Required numeric vector or $T \times k$ matrix.
Transfer rate	$\tilde{\lambda}$	`lam`	Required scalar in $(0, 1)$.
Discount factors	$(\delta_\zeta, \boldsymbol{\delta}_{\boldsymbol{\psi}})$	`tf.df`	Required vector: length 1 (shared); length 2 (δ_ζ and shared $\boldsymbol{\delta}_{\boldsymbol{\psi}}$); or length $k+1$ (componentwise).
Prior mean	$\mathbf{m}_0^{tf}$	`tf.m0`	Optional vector of length $k+1$; default $\mathbf{0}_{k+1}$.
Prior covariance	$\mathbf{C}_0^{tf}$	`tf.C0`	Optional $(k+1) \times (k+1)$ covariance matrix; default $\mathbf{I}_{k+1}$.

Table 4: Transfer-function inputs for `exdqlmTransferLDVB()` and `exdqlmTransferMCMC()`. Here k denotes the number of transfer covariates.

2.5. Special cases, including static quantile regression

The first special case of our exDQLM arises from the fact that the exAL is an extension of the AL (Yan *et al.* 2025). When $\gamma = 0$, the observational error ϵ_t in Equation (1) reduces such that $\epsilon_t \sim \text{exAL}_{p_0}(\epsilon_t|0, \sigma, \gamma) = \text{AL}_{p_0}(\epsilon_t|0, \sigma)$. This special case with observational errors distributed independently from an AL is the dynamic quantile linear model (DQLM) presented in Gonçalves, Migon, and Bastos (2017). The DQLM can be implemented with our package using the parameter `dqlm.ind = TRUE`.

Non-time-varying quantile regression is also a special case of our methodology. More specifically, when discount factors are set to 1, the evolution covariance $\mathbf{W}_t$ reduces to $\mathbf{0}$ resulting in non-time-varying parameters (West *et al.* 1985). Thus, the model reduces to the static exAL regression methodology of Yan *et al.* (2025). The static exAL special case is implemented directly in the package. This model can be implemented using our functions `exalStaticMCMC()` or `exalStaticLDVB()`, which provide MCMC and VB algorithms, respectively.

The final special case is the intersection of the previous two. A static quantile regression model with observational errors distributed according to an AL corresponds to Bayesian linear quantile regression under the AL working likelihood, first presented in Yu and Moyeed (2001). This methodology, utilized to create the Bayesian package for quantile regression **bayesQR**, can also be implemented with our package using functions `exalStaticMCMC()` or `exalStaticLDVB()` with parameter `al.ind = TRUE` (a static convenience alias of `dqlm.ind = TRUE`), which fixes $\gamma = 0$.

Examples of these special cases can be found in Section 4. A static quantile regression example specifically can be found in Section 4.4. For further details on the methodology outlined in this section, see Barata *et al.* (2022).

3. Model Diagnostics and Forecasting

To evaluate the quantile inference and predictive performance of the exDQLM, several quantitative and visual model diagnostics are included in the package.

The one-step-ahead predictive distribution function introduced by Rosenblatt (1952) is a useful diagnostic for models in which the marginal forecast distributions are unrecognizable or difficult to compute (Huerta, Jiang, and Tanner 2003; Prado, Molina, and Huerta 2006). For the exDQLM, this distribution is defined as

$$u_t = \Pr(Y_t \leq y_t | y_{1:t-1}, v_{1:T}, s_{1:T}, \sigma, \gamma), \tag{11}$$

with $y_{1:t-1} = y_1, \ldots, y_{t-1}$, $v_{1:T} = (v_1, \ldots, v_T)$ and $s_{1:T} = (s_1, \ldots, s_T)$. Here u_t defines an independent sequence which is uniformly distributed on the interval $(0, 1)$ (Rosenblatt 1952). Conditional on $\{v_{1:T}, s_{1:T}, \sigma, \gamma\}$, the forecast distribution of y_t within the exDQLM is normally distributed. Therefore, conditional on MAP estimates $\{\hat{v}_{1:T}, \hat{s}_{1:T}, \hat{\sigma}, \hat{\gamma}\}$ we estimate the sequence as

$$\hat{u}_t = \Phi(y_t | y_{1:t-1}, \hat{v}_{1:T}, \hat{s}_{1:T}, \hat{\sigma}, \hat{\gamma}) \tag{12}$$

where Φ denotes the normal CDF.

Using this estimated sequence $\{\hat{u}_t\}$, we can assess the model performance in several ways. First, we visually examine the correlation of the estimated sequence via an ACF plot. Second, by transforming the values with a standard normal inverse CDF, we compare their distributional shape to that of a standard normal with a normal QQ-plot. To quantify the divergence of this transformed sequence from normality we use the KL divergence (Kullback and Leibler 1951), $\mathrm{KL}(h, \phi) = \int_{-\infty}^{\infty} h(x) \log \frac{h(x)}{\phi(x)} \mathrm{d}x$, where h denotes the diagnostic sample distribution and ϕ is the standard normal density. The package reports nearest-neighbor estimates of this divergence by comparing the transformed forecast-error sample with a standard-normal reference sample, together with a flipped version that reverses the two arguments. Smaller

values suggest better calibration. Lastly, the MAP standardized forecast errors,

$$\frac{y_t - \mathsf{E}[y_t|y_{1:t-1}, \hat{v}_{1:T}, \hat{s}_{1:T}, \hat{\sigma}, \hat{\gamma}]}{\sqrt{\mathrm{Var}(y_t|y_{1:t-1}, \hat{v}_{1:T}, \hat{s}_{1:T}, \hat{\sigma}, \hat{\gamma})}} \tag{13}$$

can also be used as an additional graphical aid in examining the distribution at specific time points. These MAP standardized forecast errors are included in the output of the `exdqlmMCMC()` and `exdqlmLDVB()` functions.

In addition to the calibration-based diagnostics above, we compute the mean continuous ranked probability score (CRPS) from the posterior predictive draws (Gneiting and Raftery 2007). Let $\rho_p(x) = x[p - I(x < 0)]$ denote the check loss at level p. For a predictive distribution G with quantile function G^{-1} and an observation y, the CRPS is

$$\mathrm{CRPS}(y, G) = \mathbb{E}_{Y\sim G}|Y - y| - \frac{1}{2}\mathbb{E}_{Y,Y'\sim G}|Y - Y'| = 2\int_0^1 \rho_p\Big(y - G^{-1}(p)\Big)\, dp,$$

which shows that CRPS integrates check loss across all quantile levels and therefore evaluates the full predictive distribution rather than a single target level p_0. Smaller CRPS values indicate better calibrated and sharper predictive performance. For deterministic forecasts, CRPS reduces to the absolute error, so the sample mean CRPS equals the MAE.

For a final diagnostic, we include the Gelfand and Ghosh (1998) posterior predictive loss criterion (PPLC), returned by the package as `pplc`, which evaluates the posterior predictive distribution under the target-level check loss ρ_{p_0}. That is,

$$\mathrm{PPLC} = \sum_t \mathsf{E}[\rho_{p_0}(y_t^{obs} - y_t^{rep})|y_{1:T}], \tag{14}$$

where the expectation is with respect to the posterior predictive distribution of y_t, $p(y_t^{rep}|y_{1:T})$. This distribution is used to generate a replicate dataset and is the expected value of the observation equation (1) with respect to the posterior distributions of the parameters. Thus, the CRPS summarizes the full predictive distribution through an integrated check loss, whereas PPLC keeps the target quantile explicit through ρ_{p_0}. Smaller PPLC values suggest a superior model fit. Samples from the posterior replicate distributions are included in the output of the `exdqlmMCMC()` and `exdqlmLDVB()` functions.

The function `exdqlmDiagnostics()` implements the model checking criteria discussed in this section. When an output from either `exdqlmMCMC()` or `exdqlmLDVB()` is passed to `exdqlmDiagnostics()`, the resulting object includes: the estimated sequence $\{\hat{u}_t\}$, the KL divergence and flipped KL divergence, the mean CRPS, `pplc`, the ordered pairs of the QQ-plot, and autocorrelations by lag. By default, the function also produces the QQ-plot, ACF plot, and MAP standardized forecast-error plot. Examples of the utility of `exdqlmDiagnostics()` can be found in Section 4.

3.1. k-step-ahead forecast

For an arbitrary time $\tilde{t}$, the k-step-ahead future marginal distribution of the quantile is

$$\mathbf{F}'_{\tilde{t}+k}\boldsymbol{\theta}_{\tilde{t}+k}|y_1, \ldots, y_{\tilde{t}} \sim \mathrm{N}(\mathbf{F}'_{\tilde{t}+k}a_{\tilde{t}}(k), \mathbf{F}'_{\tilde{t}+k}\mathbf{R}_{\tilde{t}}(k)\mathbf{F}_{\tilde{t}+k}) \tag{15}$$

where $\mathbf{a}_{\tilde{t}}(k) = \mathbf{G}_{\tilde{t}+k}\mathbf{a}_{\tilde{t}}(k-1)$, $\mathbf{R}_{\tilde{t}}(k) = \mathbf{G}_{\tilde{t}+k}\mathbf{R}_{\tilde{t}}(k-1)\mathbf{G}'_{\tilde{t}+k} + \mathbf{W}_{\tilde{t}+k}$, $\mathbf{a}_{\tilde{t}}(0) = \mathbf{m}_{\tilde{t}}$, and $\mathbf{R}_{\tilde{t}}(0) = \mathbf{C}_{\tilde{t}}$, with $\mathbf{m}_{\tilde{t}}$ and $\mathbf{C}_{\tilde{t}}$ denoting the filtered mean and covariance of $\boldsymbol{\theta}_{\tilde{t}}$, respectively. This

distribution is implemented in the function `exdqlmForecast()`, which is compatible with the outputs from both the MCMC and VB algorithms. The fitted object supplies the filtered posterior summaries at the forecast origin, while optional future observation and evolution matrices are supplied when the required forecast design is not already stored in the fitted model. Table 5 summarizes the forecast inputs and optional posterior predictive draw controls. The output of `exdqlmForecast()` includes the parameters of the forecasted distribution as well as $\mathbf{a}_{\tilde{t}}(k)$ and $\mathbf{R}_{\tilde{t}}(k)$ for the k steps, with optional posterior predictive draw matrices available when requested.

Quantity	Mathematical symbol	R argument	Input/default
Fitted dynamic model	$(\mathbf{m}_{\tilde{t}}, \mathbf{C}_{\tilde{t}})$	`m1`	Required fitted dynamic object from `exdqlmMCMC()`, `exdqlmLDVB()`, or legacy `exdqlmISVB()`.
Forecast origin	$\tilde{t}$	`start.t`	Required integer time index within the fitted model.
Forecast horizon	k	`k`	Required positive integer number of forecast steps.
Future observation vectors	$\mathbf{F}_{(\tilde{t}+1):(\tilde{t}+k)}$	`fFF`	Optional $q \times 1$ or $q \times k$ matrix; required with `fGG` when forecasting beyond the stored model matrices.
Future evolution matrices	$\mathbf{G}_{(\tilde{t}+1):(\tilde{t}+k)}$	`fGG`	Optional $q \times q$ matrix or $q \times q \times k$ array; required with `fFF` when forecasting beyond the stored model matrices.
Credible interval mass	$1 - \alpha$	`cr.percent`	Optional scalar in $(0, 1)$; default `0.95`.
Forecast draws	$Y^{rep}_{\tilde{t}+1:\tilde{t}+k}$	`return.draws`; `n.samp`; `seed`	Optional controls; default `return.draws = FALSE`. When requested, `n.samp` sets the number of draw columns and `seed` makes generation reproducible.

Table 5: Forecast notation and main `exdqlmForecast()` inputs. Here q denotes the state dimension.

4. Examples

We demonstrate the general workflow and key features of the package through several analyses on real datasets. The first three examples illustrate dynamic workflows, while the fourth illustrates the static exAL special case. Table 6 provides an overview of the **exdqlm** functions and the resulting objects. The functions in Table 6 cover the high-level modeling workflow used in the examples. The package also exposes distribution-level utilities for the exAL error family: `dexal()`, `pexal()`, `qexal()`, and `rexal()` evaluate the density, distribution function, quantile function, and random generator, respectively, and `get_gamma_bounds()` returns the admissible interval for γ at a specified p_0. These utilities are useful for simulation, prior calibration, and direct checks of the exAL distribution, but are not needed for the main fitting, forecasting, diagnostic, or synthesis workflows illustrated below. All figures and tables are generated from scripts distributed with the article repository. These scripts build the example artifacts directly from the current **exdqlm** API. The accompanying reproducibility materials record the computing environment and outputs used in the paper.

We begin by loading the **exdqlm** package used for all examples.

Function	Returned object	Role in the workflow
Dynamic model construction		
`polytrendMod()`	`exdqlm`	Create polynomial trend component.
`seasMod()`	`exdqlm`	Create Fourier-form seasonal or periodic component.
`regMod()`	`exdqlm`	Create regression component from covariates.
`as.exdqlm()`	`exdqlm`	Coerce a compatible list or time-invariant `dlm` object.
Dynamic model fitting – Input `exdqlm` objects		
`exdqlmMCMC()`	`exdqlmMCMC`	Fit dynamic exDQLM/DQLM via MCMC.
`exdqlmLDVB()`	`exdqlmLDVB`	Fit dynamic exDQLM/DQLM via LDVB.
`exdqlmTransferLDVB()`	`exdqlmLDVB`	Fit transfer-function exDQLM via LDVB.
`exdqlmTransferMCMC()`	`exdqlmMCMC`	Fit transfer-function exDQLM via MCMC.
Static model fitting – Input design matrix and response		
`exalStaticLDVB()`	`exalStaticLDVB`	Fit static exAL/AL regression via LDVB.
`exalStaticMCMC()`	`exalStaticMCMC`	Fit static exAL/AL regression via MCMC.
Dynamic fit examination – Input `exdqlmMCMC`, `exdqlmLDVB`, or legacy `exdqlmISVB` objects		
`exdqlmForecast()`	`exdqlmForecast`	Compute forecast quantiles and optional predictive draws.
`exdqlmDiagnostics()`	`exdqlmDiagnostic`	Compute calibration and predictive diagnostics; optionally plot them.
`exdqlmPlot()`	Invisible `list`	Plot dynamic quantile MAP and CrI summaries.
`compPlot()`	Invisible `list`	Plot component or state-level MAP and CrI summaries.
Static fit examination – Input `exalStaticMCMC` or `exalStaticLDVB` objects		
`exalStaticDiagnostics()`	`exalStaticDiagnostic`	Compute static fitted-quantile diagnostics and comparisons.
Predictive synthesis – Input dynamic fit/forecast objects or draw matrices		
`quantileSynthesis()`	`exdqlmSynthesis`	Synthesize posterior predictive draws across separately fitted quantile levels.

Table 6: Core workflow functions supporting the examples. Fitting, diagnostic, forecast, static, and synthesis routines return the S3 objects shown; `exdqlmPlot()` and `compPlot()` return invisible summary lists.

```
R> library(exdqlm)
```

4.1. Lake Huron

In this example, we consider the Lake Huron time series from the package **datasets**. The data are annual measurements of the level (ft) of Lake Huron from 1875 to 1972, shown in Figure 2. With this example we show how to: build a basic state-space structure, specify a prior on γ, run the MCMC algorithm, plot the estimated quantiles and forecast distributions.

To estimate the dynamic distribution of the data, we consider three quantiles, $p_0 = 0.95, 0.50,$ and 0.05. We begin by creating the state-space structure and prior used to estimate the quantiles (i.e. $\mathbf{F}_t, \mathbf{G}_t, \mathbf{m}_0$, and $\mathbf{C}_0$ discussed in Section 2). We choose to model the quantiles with a second order polynomial trend, which we construct with the `polytrendMod()` function.

```
R> model = polytrendMod(order = 2, m0 = c(mean(LakeHuron),0), C0 = 10*diag(2))
R> model

Prior mean of the state vector:
$m0
         [,1]
[1,] 579.0041
[2,]   0.0000


Prior covariance of the state vector:
$C0
     [,1] [,2]
[1,]   10    0
[2,]    0   10


Observational vector:
$FF
     [,1]
[1,]    1
[2,]    0


Evolution matrix:
$GG
     [,1] [,2]
[1,]    1    1
[2,]    0    1
```

To allow variability in the dynamic quantile, we select a single discount factor of 0.9 to define the evolution covariance matrix $\mathbf{W}_t$. Discount factors can also be optimized, an example of which we will show in Section 4.2. We place truncated Student-t priors on the skewness parameters via `PriorGamma` as discussed in Section 2.1. In the fits shown here we use `n.burn = 2000` and `n.mcmc = 3000`. For the final fit shown above ($p_0 = 0.05$), we display R progress updates every 1000 iterations using `verbose = TRUE` and `verbose.every = 1000`. The first lines of the console output report the active MCMC and GIG-sampling backends; the global backend options that control this routing are summarized in Appendix Table 10.

```
R> set.seed(20260501)
R> M95 = exdqlmMCMC(y = LakeHuron, p0 = 0.95, model = model,
+                   df = 0.9, dim.df = 2,
+                   PriorGamma = list(m_gam = -1, s_gam = 0.1, df_gam = 1),
+                   n.burn = 2000, n.mcmc = 3000, verbose = FALSE)
R> M50 = exdqlmMCMC(y = LakeHuron, p0 = 0.50, model = model,
+                    df = 0.9, dim.df = 2,
+                    PriorGamma = list(m_gam = 0, s_gam = 0.1, df_gam = 1),
+                    n.burn = 2000, n.mcmc = 3000, verbose = FALSE)
R> M5 = exdqlmMCMC(y = LakeHuron, p0 = 0.05, model = model,
+                   df = 0.9, dim.df = 2,
+                   PriorGamma = list(m_gam = 1, s_gam = 0.1, df_gam = 1),
+                   n.burn = 2000, n.mcmc = 3000, verbose = TRUE, verbose.every = 1000)

MCMC backend: C++ (mode=fast)
running LDVB algorithm to initialize mcmc
```

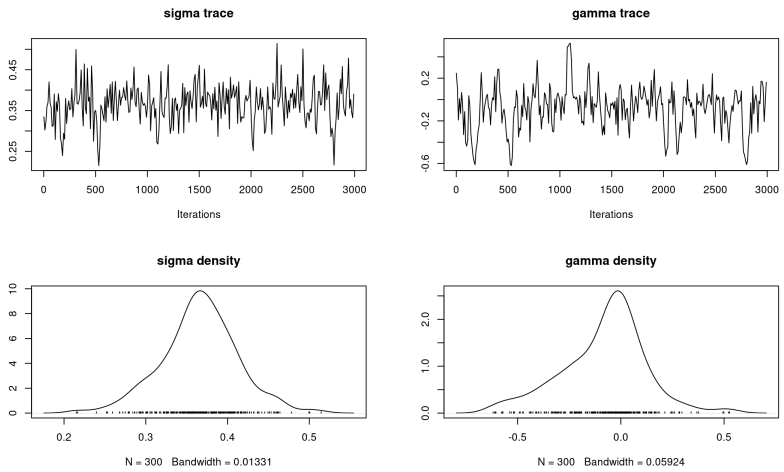

Figure 1: Example 1: Lake Huron. Trace and density plots from the median run with 2000 burn-in iterations and 3000 retained draws. For readability, every 10th retained draw is used in the plotting step. The top row shows the posterior trace plots for σ and γ, and the bottom row shows the corresponding posterior densities.

```
GIG backend: C++ Devroye (required)
MCMC start | burn=2000.00 | keep=3000.00 | kernel=slice | warm_start=ldvb
MCMC progress | model=exDQLM | phase=burn | iter=1000/5000 | kept=0/3000
MCMC progress | model=exDQLM | phase=burn | iter=2000/5000 | kept=0/3000
MCMC progress | model=exDQLM | phase=keep | iter=3000/5000 | kept=1000/3000
MCMC progress | model=exDQLM | phase=keep | iter=4000/5000 | kept=2000/3000
MCMC progress | model=exDQLM | phase=keep | iter=5000/5000 | kept=3000/3000
MCMC done | model=exDQLM | status=complete | iter=5000.00
```

By default, `exdqlmMCMC()` uses the package's VB routine, `exdqlmLDVB()`, to warm-start the chain. This initialization choice is separate from the subsequent MCMC kernel. Because the data are relatively symmetric, we do not expect the skewness parameter γ to be far from zero at $p_0 = 0.5$. Consistent with this, the posterior mean of γ is -0.012 with a 95% credible interval $(-0.429, 0.462)$, while the posterior mean of σ is 0.369 with a 95% credible interval $(0.282, 0.468)$. All retained samples in the output of `exdqlmMCMC()` are `mcmc` objects, so we can use the package **coda** to examine the trace and density plots of both σ and γ. For readability, we plot every 10th retained draw in Figure 1.

```
R> keep.idx = seq(1, length(M50$samp.sigma), by = 10)
R> sigma.trace = coda::mcmc(M50$samp.sigma[keep.idx], thin = 10)
R> gamma.trace = coda::mcmc(M50$samp.gamma[keep.idx], thin = 10)
R> coda::traceplot(sigma.trace, main = expression(sigma), ylab="trace")
R> coda::densplot(sigma.trace, main = "", ylab = "density")
R> coda::traceplot(gamma.trace, main = expression(gamma))
R> coda::densplot(gamma.trace, main = "")
```

The posterior summary indicates that γ is not distinguishable from zero, so the additional flexibility of the skewness parameter is not needed for the median fit. We therefore re-run the model with a point-mass prior on γ at 0 using the settings `gam.init = 0` and `fix.gamma`

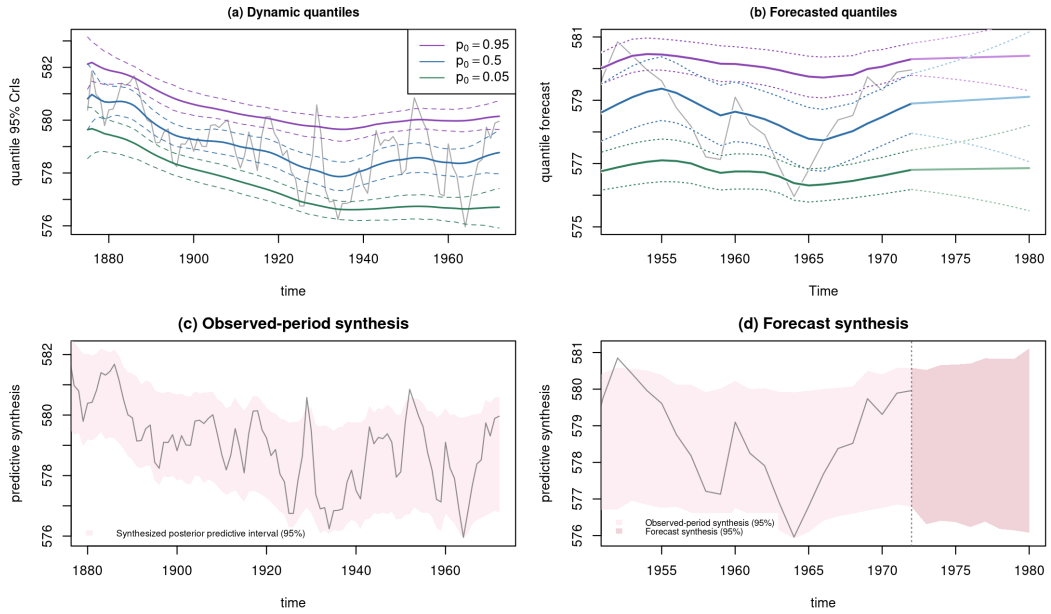

Figure 2: Example 1: Lake Huron. Top-left: MAP estimates and 95% CrIs of the estimated quantiles, plotted with the data (grey). Top-right: forecasted quantile estimates and 95% credible intervals (seen after 1972), along with the filtered quantile estimates and 95% credible intervals (seen from 1952 to 1972). Bottom-left: synthesized posterior predictive 95% interval over the observed period. Bottom-right: observed-period synthesis near the forecast origin (lighter band) and synthesized posterior predictive 95% interval over the eight-step forecast window (slightly darker rose band); the vertical dotted line marks the forecast origin.

`= TRUE`, which is equivalent to the setting `dqlm.ind = TRUE`. This reduces the model to the DQLM mentioned in Section 2.5.

```
R> M50 = exdqlmMCMC(y = LakeHuron, p0 = 0.50, model = model,
+                   df = 0.9, dim.df = 2,
+                   gam.init = 0, fix.gamma = TRUE,
+                   n.burn = 2000, n.mcmc = 3000)
```

The results are `exdqlmMCMC` objects that can be used for analysis and forecasting. First we examine the estimated quantiles with the plot method for `exdqlmMCMC` objects, seen in the top-left panel of Figure 2. This generates a plot of the data with the MAP estimates and the 95% CrIs of the dynamic quantile. Subsequent quantiles (i.e. $p_0 = 0.50, 0.05$ in this example) can be added to the plot using the function `exdqlmPlot()` with the argument `add = TRUE`.

```
R> plot(M95); title("Dynamic quantiles")
R> exdqlmPlot(M50, add = TRUE, col = "blue")
R> exdqlmPlot(M5, add = TRUE, col = "forest green")
R> legend("topright", lty = 1, bty = "n", col = c("purple","blue","forest green"),
+        legend = c(expression('p'[0]*'=0.95'),expression('p'[0]*'=0.50'),
+                   expression('p'[0]*'=0.05')))
```

Next, we forecast the $k = 8$ step-ahead distributions starting in 1972. To do this, we first create the observational vector (`fFF`) and evolution matrix (`fGG`) that will be used in the

forecast updates. In this case both are non-time-varying and identical to those used to estimate the quantile.

```
R> fFF = model$FF
R> fGG =  model$GG
```

We plot the time series data in a narrower range for closer examination and add the forecast estimates using `exdqlmForecast()` with argument `add = TRUE`. Notice we start the forecast at the last time index of the data, i.e. `start.t = length(LakeHuron)`.

```
R> plot(LakeHuron,  xlim = c(1952,1980), ylim = c(575,581),
+        col = "dark grey", main = "Forecasted quantiles",
+           ylab = "forecast 95% CrIs")
R> fc95 = exdqlmForecast(start.t = length(LakeHuron), k = 8, m1 = M95,
+               fFF = fFF, fGG = fGG, plot = TRUE, add = TRUE,
+               return.draws = TRUE)
R> fc50 = exdqlmForecast(start.t = length(LakeHuron), k = 8, m1 = M50,
+               fFF = fFF, fGG = fGG, plot = TRUE, add = TRUE,
+               cols = c("blue","light blue"), return.draws = TRUE)
R> fc05 = exdqlmForecast(start.t = length(LakeHuron), k = 8, m1 = M5,
+               fFF = fFF, fGG = fGG, plot = TRUE, add = TRUE,
+               cols = c("forest green","green"), return.draws = TRUE)
```

This generates a plot of the data with the forecasted quantile estimates and 95% credible intervals (seen after 1972), along with the filtered quantile estimates and 95% credible intervals (before 1972), for reference. Results can be seen in the top-right panel of Figure 2. The percentage in the CrIs can be modified with the parameter `cr.percent`.

When several quantile models are fitted separately, the package also allows their posterior predictive draws to be combined into a single coherent predictive distribution through `quantileSynthesis()`. Each fitted model contributes local predictive information near its target quantile, and the synthesis step interpolates across these quantile-specific draws to obtain one posterior predictive distribution. When independently fitted quantiles induce crossing, optional monotonicity corrections based on isotonic adjustment and monotone rearrangement are available (Barlow, Bartholomew, Bremner, and Brunk 1972; Chernozhukov, Fernández-Val, and Galichon 2010). For the Lake Huron example, we apply this routine directly to the fitted objects `M5`, `M50`, and `M95` over the observed period.

```
R> syn.obs = quantileSynthesis(
+               draws_list = list(M5, M50, M95),
+               p = c(0.05, 0.50, 0.95),
+               T_expected = length(LakeHuron))
```

Because the argument `return.draws = TRUE` was used when producing the forecast objects (`fc05`, `fc50`, `fc95`) above, each object stores posterior predictive forecast draws in the return value `samp.fore`. This allows us to apply the same synthesis step to the eight-step forecast horizon.

```
R> syn.fore = quantileSynthesis(
+               draws_list = list(fc05, fc50, fc95),
+               p = c(0.05, 0.50, 0.95),
+               T_expected = 8)
```

The objects `syn.obs` and `syn.fore` have class `exdqlmSynthesis`, so the corresponding `plot()` method can be used to display the synthesized posterior predictive intervals. The lower panels of Figure 2 are produced from these objects as follows. In the forecast panel, we use a slightly darker related fill for the forecast synthesis and draw a small visual bridge from the final observed-period synthesis point to the first one-step-ahead forecast point so that the observed and forecast intervals are displayed on the same annual time scale.

```
R> synth.obs.col = adjustcolor("#F7D6DE", alpha.f = 0.40)
R> synth.fore.col = adjustcolor("#D98A9B", alpha.f = 0.38)
R> t.end = tail(time(LakeHuron), 1)
R> t.fore = seq(t.end + 1, t.end + 8)
R> plot(syn.obs, y = LakeHuron, time = time(LakeHuron),
+       xlim = c(1952, t.end), ylab = "predictive synthesis",
+       show.median = FALSE, band.col = synth.obs.col)
R> plot(syn.obs, y = LakeHuron, time = time(LakeHuron),
+       xlim = c(1952, max(t.fore)), ylab = "predictive synthesis",
+       show.median = FALSE, band.col = synth.obs.col)
R> polygon(c(t.end, t.fore[1], t.fore[1], t.end),
+          c(tail(syn.obs$summary$q025, 1), syn.fore$summary$q025[1],
+            syn.fore$summary$q975[1], tail(syn.obs$summary$q975, 1)),
+          col = synth.fore.col, border = NA)
R> plot(syn.fore, time = t.fore, add = TRUE,
+       show.median = FALSE, band.col = synth.fore.col)
R> abline(v = t.end, lty = 3)
```

4.2. Sunspots

For our next example, we use the yearly Sunspot time series from the package **datasets**. The data are yearly counts for sunspots from 1700 to 1988, shown in Figure 3. In this example we show how to: use the **dlm** package to create the state-space model, combine blocks of a state-space model, apply the VB approximation through `exdqlmLDVB()`, perform visual model diagnostics to compare the exDQLM with the DQLM, and use those diagnostics for discount-factor selection.

The solar cycle impacts astronauts in space as well as lives and technology on earth. Of particular interest to scientists is the solar max, or period in which the sun is most active Fox (2012), therefore we will explore the 0.85 quantile in this example. Again, we begin by building the state-space structure used to estimate the quantile. Here we choose to model the quantile with a first order polynomial trend component as well as a Fourier form seasonal component that incorporates a fundamental period every 11 observations (years), as well as some of its corresponding harmonics. This results in a dynamic model with a total of 9 state parameters, 1 for the first order polynomial component and 8 for the seasonal component. The **exdqlm** functions are compatible with time-invariant `dlm` objects from the **dlm** package. For example, we create the first order trend component with the function `dlmModPoly()` from the **dlm** package, then use `as.exdqlm()` to turn the output into an `exdqlm` object.

```
R> dlm.trend.comp = dlm::dlmModPoly(1, m0 = mean(sunspot.year), C0 = 10)
R> trend.comp = as.exdqlm(dlm.trend.comp)
```

Next, we construct the seasonal component with our `seasMod` function. The duration of sunspot cycles typically follow a period of 11 years, of which we include the first 4 harmonics in the component.

```
R> seas.comp = seasMod(p = 11, h = 1:4, C0 = 10*diag(8))
```

To combine the two components into a single state-space structure, we add the two `exdqlm` objects.

```
R> model = trend.comp + seas.comp
```

The resulting evolution matrix of the combined models is printed for illustration.

```
R> model$GG
```

```
     [,1]    [,2]   [,3]    [,4]   [,5]    [,6]    [,7]    [,8]    [,9]
 [1,]    1  0.0000 0.0000  0.0000 0.0000  0.0000  0.0000  0.0000  0.0000
 [2,]    0  0.8413 0.5406  0.0000 0.0000  0.0000  0.0000  0.0000  0.0000
 [3,]    0 -0.5406 0.8413  0.0000 0.0000  0.0000  0.0000  0.0000  0.0000
 [4,]    0  0.0000 0.0000  0.4154 0.9096  0.0000  0.0000  0.0000  0.0000
 [5,]    0  0.0000 0.0000 -0.9096 0.4154  0.0000  0.0000  0.0000  0.0000
 [6,]    0  0.0000 0.0000  0.0000 0.0000 -0.1423  0.9898  0.0000  0.0000
 [7,]    0  0.0000 0.0000  0.0000 0.0000 -0.9898 -0.1423  0.0000  0.0000
 [8,]    0  0.0000 0.0000  0.0000 0.0000  0.0000  0.0000 -0.6549  0.7557
 [9,]    0  0.0000 0.0000  0.0000 0.0000  0.0000  0.0000 -0.7557 -0.6549
```

We will apply discount factors by component, i.e. a discount factor of 0.9 for the trend component of dimension 1 and a discount factor of 0.85 for the seasonal component of dimension 8. This discounting strategy can be applied with arguments `df = c(0.9,0.85)` and `dim.df = c(1,8)`.

We call the function `exdqlmLDVB()` to obtain the VB approximations. We use the routine with argument `dqlm.ind = TRUE` to estimate the DQLM, as well as with the default settings to estimate the exDQLM. In this example we set `n.samp = 3000`.

```
R> M1 = exdqlmLDVB(y = sunspot.year, p0 = 0.85, model = model,
+                  df = c(0.9,0.85), dim.df = c(1,8),
+                  dqlm.ind = TRUE, fix.sigma = FALSE,
+                  n.samp = 3000, verbose = FALSE)
R> M2 = exdqlmLDVB(y = sunspot.year, p0 = 0.85, model = model,
+                  df = c(0.9,0.85), dim.df = c(1,8),
+                  fix.sigma = FALSE,
+                  n.samp = 3000, verbose = FALSE)
```

The output is suppressed using input `verbose = FALSE`, however we check the run-time of each model after fitting using the `summary()` generic function. For this dataset of length 289, the DQLM VB fit completes quickly, while the corresponding exDQLM VB fit is more computationally demanding because the skewness parameter is estimated rather than fixed.

```
R> summary(M1)
```

```
Bayesian Dynamic Quantile Regression Model (DQLM)
Number of Observations: 289
State Dimension: 9
Discount factors ( dimensions ): 0.9 ( 1 ), 0.85 ( 8 )

DQLM fitted using VB
Iterations until convergence: 69
Run-time: 6.427 seconds
```

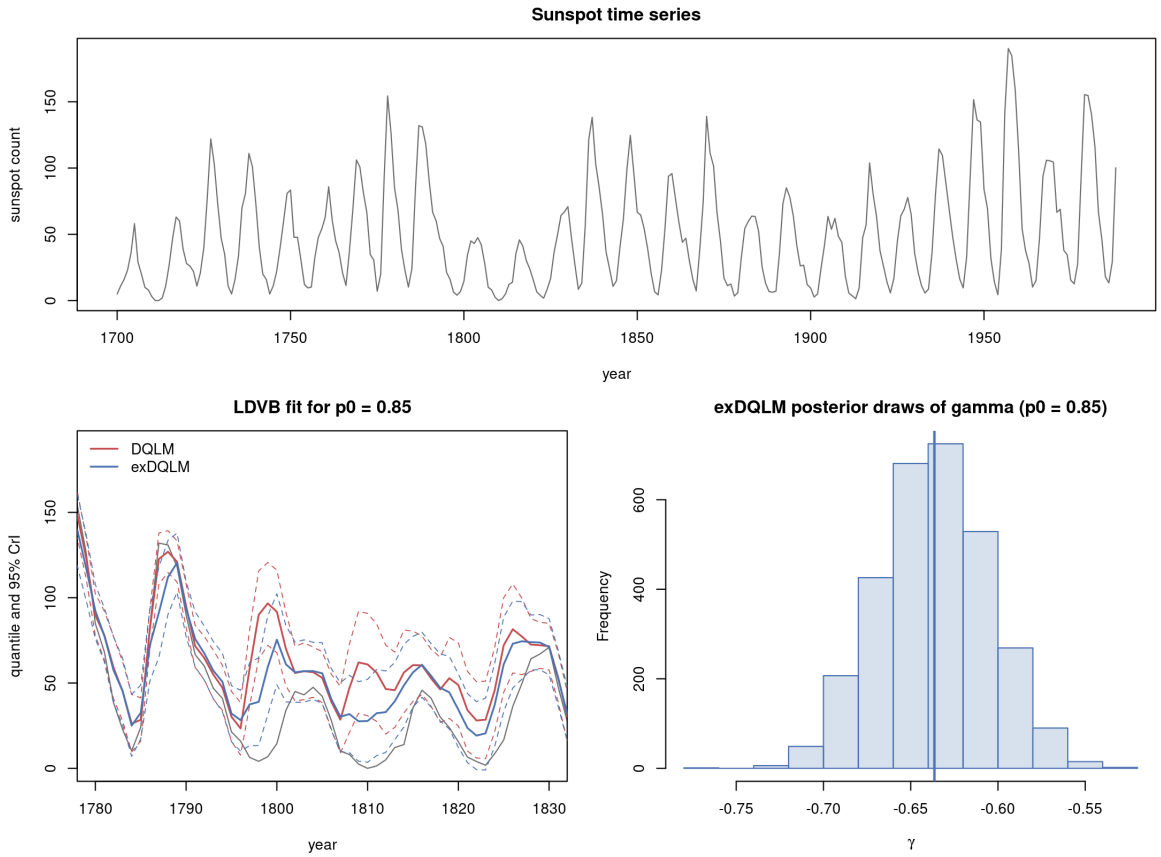

Figure 3: Example 2: Sunspots. Top: The sunspot time series from 1700 to 1988. Bottom left: MAP estimates and 95% CrIs of the estimated quantiles from the DQLM (red) and exDQLM (blue), plotted with the data (grey) from 1780 to 1830. Bottom right: Histogram of samples from the approximated posterior distribution of γ under the exDQLM.

```
R> summary(M2)

Bayesian Dynamic Quantile Regression Model (exDQLM)
Number of Observations: 289
State Dimension: 9
Discount factors ( dimensions ): 0.9 ( 1 ), 0.85 ( 8 )

exDQLM fitted using LDVB
Parameters estimated with LD: gamma sigma
Iterations until convergence: 200
Run-time: 30.589 seconds
```

We can visualize the differences between the two resulting dynamic quantiles using the function `exdqlmPlot()`, shown in the lower-left panel of Figure 3. For clarity we limit that panel to years 1780 to 1830.

```
R> plot(sunspot.year, xlim = c(1780,1830), col = "dark grey",
+       ylab = "quantile 95% CrIs")
R> exdqlmPlot(M1, add = TRUE, col = "red")
```

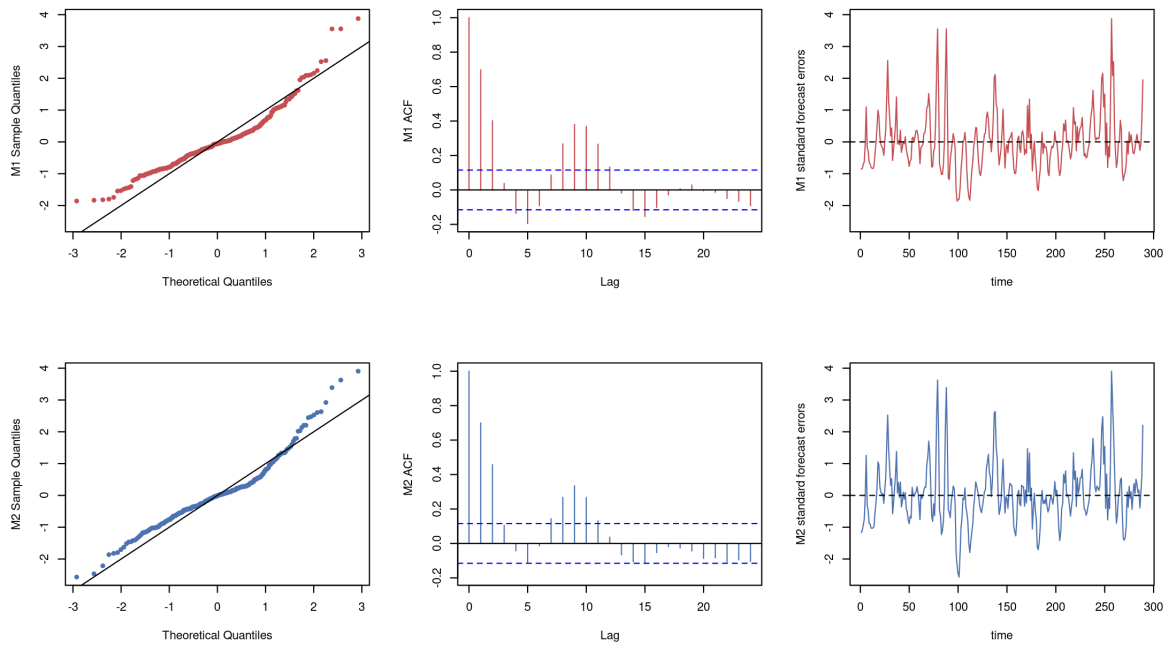

Figure 4: Example 2: Sunspots. The QQ plots (left column), ACF plots of the PIT sequence (center column), and MAP one-step-ahead standardized forecast errors (right column) from the DQLM (red) and exDQLM (blue).

```
R> exdqlmPlot(M2, add = TRUE, col = "blue")
R> legend("topleft", lty = 1, bty = "n", col = c("red","blue"),
+         legend = c("DQLM","exDQLM"))
```

To examine whether the added flexibility of the exDQLM is significant, we can also visualize the approximate posterior distribution of the skewness parameter γ by plotting the samples from the corresponding variational distribution, also seen in Figure 3. The approximate distribution is clearly distinct from zero, so the additional skewness flexibility is useful for this upper-quantile fit.

```
R> hist(M2$samp.gamma, xlab=expression(gamma), main="")
R> abline(v = mean(M2$samp.gamma), col="blue")
```

To further examine the two models, we use the function `exdqlmDiagnostics()` to plot the model diagnostics discussed in Section 3. The results are seen in Figure 4. These diagnostic plots are based on the MAP one-step-ahead standardized forecast errors produced by the filtering recursion, together with the corresponding probability integral transform (PIT) sequence. Thus the figure summarizes the fitted diagnostic sequence rather than posterior bands for residuals. For reproducibility, we fix the standard-normal reference sample used in the KL calculations and reuse it below in the discount-factor scan. The returned `exdqlmDiagnostic` object can also be passed to the `print()` generic to display the numerical comparison. Smaller values indicate better fit for the KL, CRPS, and PPLC criteria; in this LDVB comparison, all reported fit-quality diagnostics favor the exDQLM, while the DQLM is faster.

```
R> set.seed(20260503)
R> ref.samp = rnorm(length(sunspot.year))
R> diagM1M2 = exdqlmDiagnostics(M1, M2, cols = c("red","blue"),
+                               ref = ref.samp)
R> print(diagM1M2)
```

```
   Diagnostic       M1        M2
           KL    0.144    0.0782
 KL (flipped)    0.159    0.0390
         CRPS    8.616    6.4309
         pplc 3924.038 2350.4807
 run-time (s)   29.828  128.9230
```

Finally, the function `exdqlmDiagnostics()` can also be used for model selection. Say we want to check whether the discount factor we used for the seasonal component is reasonable under a predictive scoring rule. We can create a list of possible discount factors, quickly run the VB approximation for each possible model, and compare the resulting CRPS and KL values. In practice we recommend selecting the discount factors with the CRPS and using the KL divergence as a complementary calibration check. A simple example is seen below.

```
R> possible.dfs = cbind(0.9,seq(0.85,1,0.05))
R> possible.dfs
```

```
     [,1] [,2]
[1,]  0.9 0.85
[2,]  0.9 0.90
[3,]  0.9 0.95
[4,]  0.9 1.00
```

```
R> metrics <- matrix(NA_real_, nrow(possible.dfs), 2,
+                    dimnames = list(NULL, c("CRPS","KL")))
R> for(i in 1:nrow(possible.dfs)){
+   temp.M2 = exdqlmLDVB(y = sunspot.year, p0 = 0.85, model = model,
+                  df = possible.dfs[i,], dim.df = c(1,8),
+                  sig.init = 2, fix.sigma = FALSE,
+                  n.samp = 3000, verbose = FALSE)
+   temp.check = exdqlmDiagnostics(temp.M2, plot = FALSE,
+                                  ref = ref.samp)
+   metrics[i,] = c(temp.check$m1.CRPS, temp.check$m1.KL)
+ }
R> # optimal dfs based off CRPS
R> possible.dfs[which.min(metrics[, "CRPS"]),]
```

```
[1] 0.90 0.85
```

On this coarse grid, the CRPS and KL criteria agree and both select the seasonal discount factor 0.85, so the choice used above is retained.

Finally, to compare the fast variational approximation with posterior simulation, we fit matched DQLM and exDQLM specifications with `exdqlmMCMC()`. The MCMC fits use the same state-space model, quantile level, and discount factors as the LDVB fits, with 2000 burn-in iterations and 3000 retained draws.

model	method	runtime (s)	KL	CRPS	PPLC
DQLM	VB	**29.83**	0.144	8.616	3924.0
	MCMC	361.82	**0.109**	**7.655**	**3022.4**
exDQLM	VB	**128.92**	**0.078**	6.431	2350.5
	MCMC	452.56	0.157	**5.057**	**2184.2**

Table 7: Example 2: representative dynamic benchmark for the DQLM and exDQLM fits at $p_0 = 0.85$. Each row reports runtime together with the KL divergence, the mean CRPS from posterior predictive draws, and the posterior predictive loss criterion (PPLC). The VB rows use `n.samp = 3000`. The MCMC rows use 2000 burn-in iterations and 3000 retained draws. Because the columns are the comparison metrics, bold entries indicate the smaller value within each model block; displayed ties are left unbolded.

```
R> M1mcmc = exdqlmMCMC(y = sunspot.year, p0 = 0.85, model = model,
+                 df = c(0.9,0.85), dim.df = c(1,8),
+                 n.burn = 2000, n.mcmc = 3000, verbose = FALSE,
+                 dqlm.ind = TRUE, fix.sigma = FALSE)
R> M2mcmc = exdqlmMCMC(y = sunspot.year, p0 = 0.85, model = model,
+                 df = c(0.9,0.85), dim.df = c(1,8),
+                 n.burn = 2000, n.mcmc = 3000, verbose = FALSE,
+                 fix.sigma = FALSE)
```

Table 7 gives a compact runtime-and-diagnostic comparison. Within each model class, VB is substantially faster. For the simpler DQLM, MCMC gives smaller KL, CRPS, and PPLC values. For the more flexible exDQLM, VB gives the smaller KL value, while MCMC gives smaller CRPS and PPLC values. Comparing model classes within each inferential method, the exDQLM improves the LDVB diagnostics relative to the DQLM, and under MCMC it improves the predictive-score criteria CRPS and PPLC. These results reinforce the importance of reporting runtime together with predictive criteria rather than runtime alone.

4.3. Big Tree water flow

For the next example, we consider observed average monthly water flow (cubic feet per second) at the Big Tree gauge of the San Lorenzo River in Santa Cruz County, CA (U.S. Geological Survey 2016). The **exdqlm** package includes these measurements as the monthly time series `BTflow`, which extends through March 2026. In the analysis below, we use the complete overlap between `BTflow` and the package data frame `climateIndices`, giving 432 monthly observations from January 1987 through December 2022. This example illustrates how to construct a dynamic quantile model with external covariates, fit the transfer-function extension, examine fitted components, and compare models using the diagnostics introduced above.

Because low-flow behavior is the main hydrological feature of interest, we focus on the $p_0 = 0.15$ quantile. As predictors, we use two standardized monthly climate indices from `climateIndices`: the Northern Oscillation Index (NOI) and the Atlantic Multidecadal Oscillation (AMO) index. These indices were selected from a broader screen of monthly climate indices compiled from the NOAA Physical Sciences Laboratory climate-index collection (NOAA Physical Sciences Laboratory 2026). We use only two indices, rather than a larger screened set, so that the section remains focused on the package workflow. Both models are

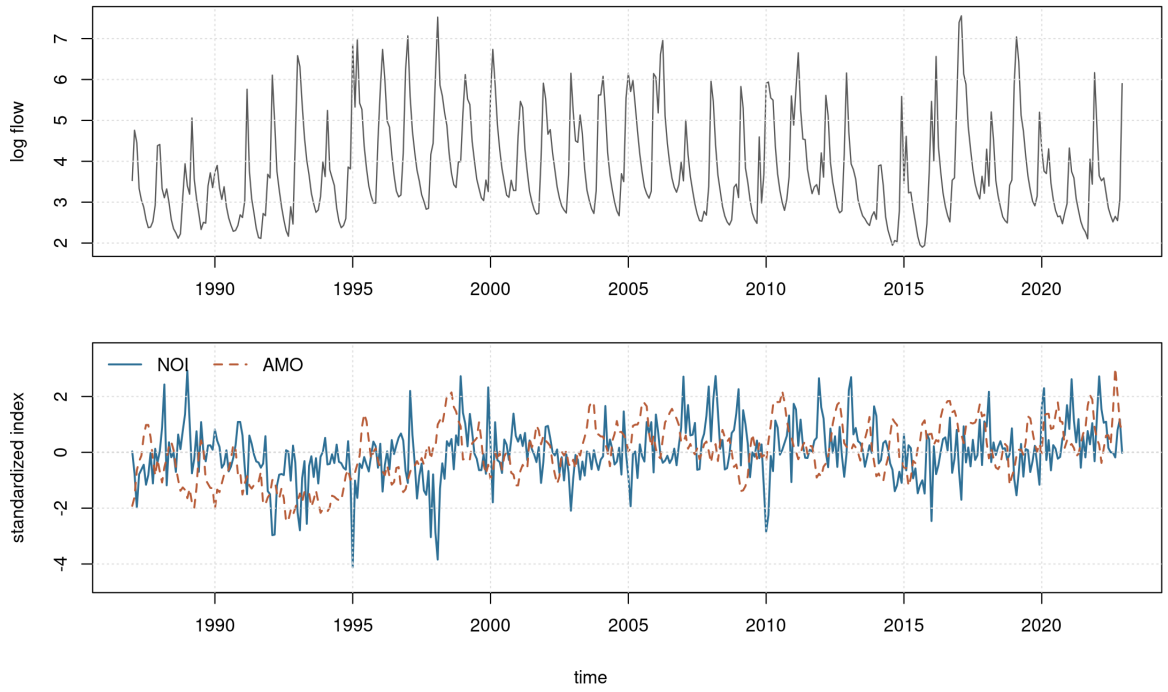

Figure 5: Example 3: Big Tree water flow. Top: observed average monthly water flow at the Big Tree gauge of the San Lorenzo River in Santa Cruz County, CA, plotted on the log scale. Bottom: standardized monthly values of the Northern Oscillation Index (NOI) and Atlantic Multidecadal Oscillation (AMO) index. The displayed modeling window spans January 1987 through December 2022.

fit to the log monthly flow series shown in Figure 5.

```
R> data("BTflow", package = "exdqlm")
R> data("climateIndices", package = "exdqlm")
R> bt.dates = seq(as.Date("1987-01-01"), by = "month",
+                 length.out = length(BTflow))
R> flow.df = data.frame(date = bt.dates, flow = as.numeric(BTflow))
R> index.df = climateIndices[, c("date", "noi", "amo")]
R> ex3.df = merge(flow.df, index.df, by = "date")
R> ex3.df = subset(ex3.df, date >= as.Date("1987-01-01") &
+                          date <= as.Date("2022-12-01"))
R> flow.fit = ts(ex3.df$flow, start = c(1987, 1), frequency = 12)
R> y.fit = log(flow.fit)
R> X.input = scale(as.matrix(ex3.df[, c("noi", "amo")]))
```

The baseline state-space model contains a first-order polynomial trend and a Fourier-form seasonal component with period 12. The seasonal block includes $h = 1$, $h = 2$, and $h \approx 0.147$, corresponding to annual, semiannual, and approximately 7-year cycles on the monthly scale.

```
R> trend.comp = polytrendMod(1, m0 = quantile(y.fit, 0.15), C0 = 0.1)
R> seas.comp = seasMod(p = 12, h = c(1, 2, 0.1469118636),
+                      C0 = diag(1,6))
R> model = trend.comp + seas.comp
```

Our first model, M_1, adds NOI and AMO as contemporaneous dynamic regression effects. Combining this regression block with the trend and seasonal blocks gives the complete state-space structure for the dynamic quantile regression model.

```
R> reg.comp = regMod(X.input, m0 = rep(0, 2), C0 = diag(1, 2))
R> model.w.reg = model + reg.comp
```

To allow the components to evolve gradually over time while keeping the fitted paths smooth, all discount factors are set to 0.99. Thus the same discounting level is used for the trend, seasonal block, dynamic regression coefficients, and transfer-function components. In the package interface, these choices are passed through `df` for the base state-space model and through `tf.df` for the transfer-function states.

The second model, M_2, uses the transfer-function structure from Section 2.4. We use the LDVB transfer-function implementation for a fast approximate fit to the monthly series; the corresponding MCMC implementation is available when posterior simulation is desired. The transfer rate λ is selected by scanning the grid $0.1, 0.2, \ldots, 0.9$ and choosing the fitted value with the smallest finite CRPS. The code below records failed fits and non-finite diagnostic values explicitly, which is useful in grid searches where some candidate specifications may be numerically unstable. In this run, all candidate values returned finite diagnostics and CRPS selected $\lambda = 0.5$.

```
R> possible.lams = seq(0.1, 0.9, 0.1)
R> metrics = data.frame(lambda = possible.lams, CRPS = NA_real_,
+                       KL = NA_real_, pplc = NA_real_, status = "not_run")
R> ref.samp = rnorm(length(y.fit))
R> for(i in seq_along(possible.lams)){
+    temp.M2 = tryCatch(
+      exdqlmTransferLDVB(y = y.fit, p0 = 0.15, model = model,
+                         df = c(0.99,0.99), dim.df = c(1,6),
+                         X = X.input, tf.df = c(0.99,0.99),
+                         lam = possible.lams[i],
+                         tf.m0 = rep(0,3),
+                         tf.C0 = diag(c(0.1,0.005,0.005),3),
+                         sig.init = 0.1, gam.init = -0.1,
+                         n.samp = 1000, tol = 0.05, verbose = FALSE),
+      error = identity)
+    if(inherits(temp.M2, "error")) {
+      metrics$status[i] = "fit_error"
+      next
+    }
+    temp.check = tryCatch(
+      exdqlmDiagnostics(temp.M2, plot = FALSE, ref = ref.samp),
+      error = identity)
+    if(inherits(temp.check, "error")) {
+      metrics$status[i] = "diagnostic_error"
+      next
+    }
+    metrics[i, c("CRPS","KL","pplc")] =
+      c(temp.check$m1.CRPS, temp.check$m1.KL, temp.check$m1.pplc)
+    metrics$status[i] = "ok"
+  }
R> ok = metrics$status == "ok" & is.finite(metrics$CRPS)
R> if(!any(ok)) stop("No finite CRPS values were produced.")
R> metrics$lambda[which(ok)[which.min(metrics$CRPS[ok])]]
```

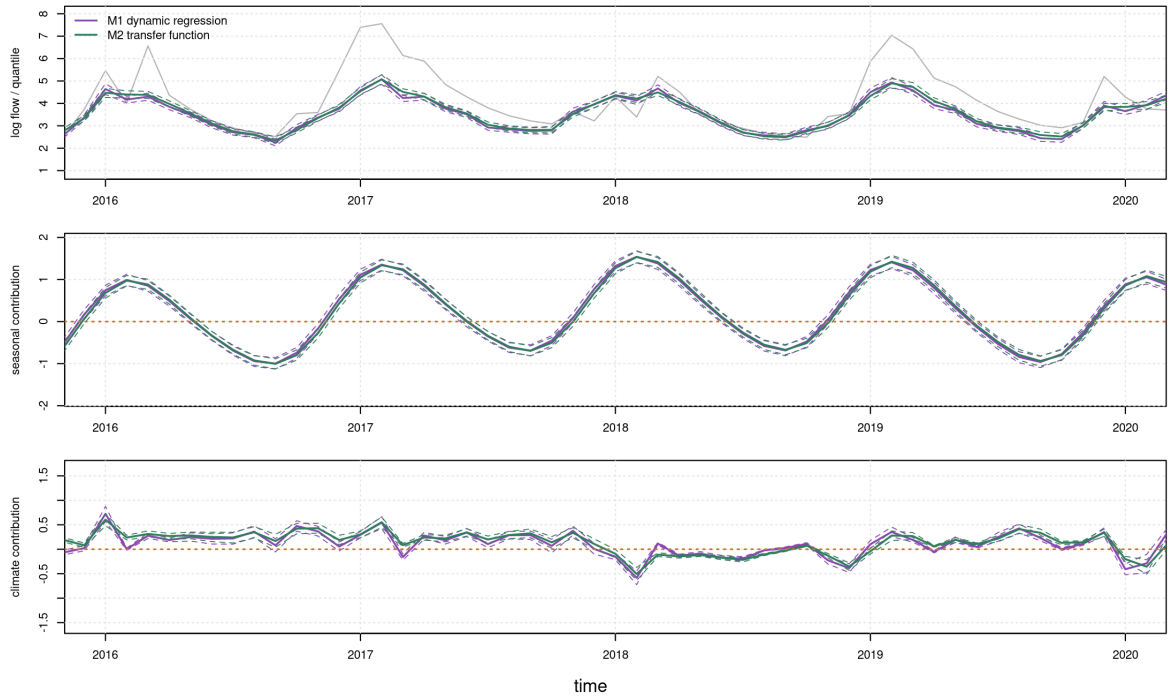

Figure 6: Example 3: Big Tree water flow. Top: MAP estimates and 95% CrIs of the estimated quantiles from the dynamic regression model (purple) and transfer-function model (green), plotted with the data (grey) from 2016 to 2020. Middle: MAP estimates and 95% CrIs of the combined seasonal components implied by the annual, semiannual, and low-frequency harmonics. Bottom: MAP estimates and 95% CrIs of the combined climate-index contribution from NOI and AMO. For M_1, the purple curve is the direct dynamic regression contribution. For M_2, the green curve is the transfer-function contribution. The horizontal orange dotted line is at zero for reference.

```
[1] 0.5
```

With these discount factors and the CRPS-selected value of λ, we estimate the dynamic regression model `M1` and the transfer-function model `M2`.

```
R> M1 = exdqlmLDVB(y = y.fit, p0 = 0.15, model = model.w.reg,
+                  df = c(0.99,0.99,0.99), dim.df = c(1,6,2),
+                  sig.init = 0.1, gam.init = - 0.1,
+                  n.samp = 1000, tol = 0.05, verbose = FALSE)
R> M2 = exdqlmTransferLDVB(y = y.fit, p0 = 0.15, model = model,
+                          df = c(0.99,0.99), dim.df = c(1,6),
+                          X = X.input, tf.df = c(0.99,0.99), lam = 0.50,
+                          tf.m0 = rep(0,3),
+                          tf.C0 = diag(c(0.1,0.005,0.005),3),
+                          sig.init = 0.1, gam.init = - 0.1,
+                          n.samp = 1000, tol = 0.05, verbose = FALSE)
```

The upper panel of Figure 6 compares the fitted 0.15 quantiles from the two models over 2016–2020, a wet period in the region that provides a more focused view of the fitted low-flow

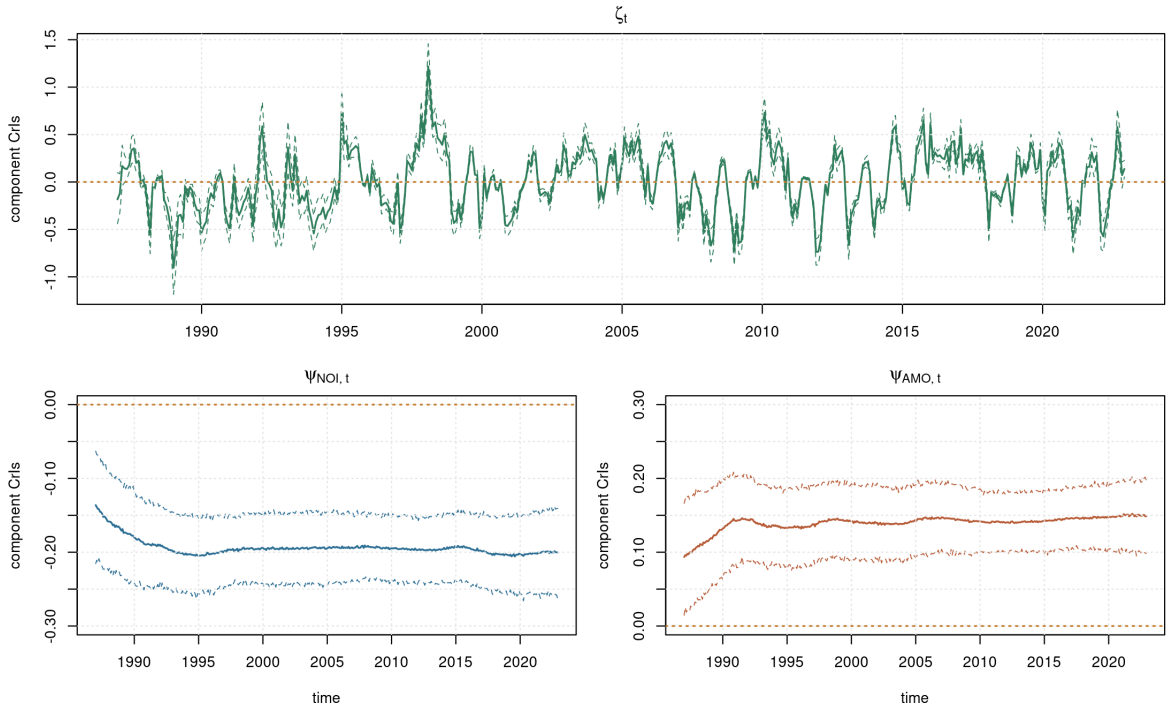

Figure 7: Example 3: Big Tree water flow. MAP estimates and 95% CrIs of the transfer-function states. The top panel shows ζ_t in green. The bottom-left panel shows $\psi_{\mathrm{NOI},t}$ in blue with y-axis range $[-0.3, 0]$, and the bottom-right panel shows $\psi_{\mathrm{AMO},t}$ in orange with y-axis range $[0, 0.3]$. Horizontal dotted orange lines mark zero.

dynamics. The package function `exdqlmPlot()` displays the MAP estimate together with pointwise 95% CrIs and can be used with `add = TRUE` to overlay fitted quantiles from several models.

```
R> plot(y.fit, col = "grey", ylim = c(1,8), xlim=c(2016,2020),
+       ylab = "quantile 95% CrIs")
R> exdqlmPlot(M1, add = TRUE)
R> exdqlmPlot(M2, add = TRUE, col = "forest green")
```

The fitted quantiles differ most clearly during periods of low flow. To see which parts of the state-space model drive these differences, we use `compPlot()` to examine selected model components. We begin with the combined seasonal contribution. Because the seasonal block contains three harmonics, this contribution is formed from the second through seventh elements of the state vector. The middle panel of Figure 6 shows the resulting seasonal estimates for the two models over the same 2016–2020 window.

```
R> plot(NA, ylim = c(-2,2), xlim = c(2016,2020),
+         ylab = "seasonal components")
R> compPlot(M1, index = 2:7, add = TRUE)
R> compPlot(M2, index = 2:7, add = TRUE, col = "forest green")
```

The lower panel of Figure 6 compares the climate-index contribution. For `M1`, this is the combined direct regression contribution of the standardized NOI and AMO covariates. For

M2, this is the transfer-function contribution induced by the same covariates. This view is useful because it separates differences in the fitted low-flow quantile into seasonal and covariate-driven components.

```
R> plot(NA, ylim = c(-2,2), xlim = c(2016,2020),
+          ylab = "climate-index components")
R> compPlot(M1, index = 8:9, add = TRUE)
R> compPlot(M2, index = 8, add = TRUE, col = "forest green")
R> abline(h = 0, col = "orange", lty = 3, lwd = 2)
```

For the transfer-function model, it is also informative to inspect the augmented transfer states directly. As described in Section 2.4, ζ_t controls the dynamic transfer effect and the ψ_t states determine how the contemporaneous covariates enter that transfer term. Setting `just.theta = TRUE` in `compPlot()` plots a single element of the state vector $\boldsymbol{\theta}_t$, or $\tilde{\boldsymbol{\theta}}_t$ for the transfer-function model. In this fitted model, indices 8, 9, and 10 correspond to ζ_t, $\psi_{\mathrm{NOI},t}$, and $\psi_{\mathrm{AMO},t}$, respectively; these states are shown in Figure 7.

```
R> layout(matrix(c(1,1,2,3), nrow = 2, byrow = TRUE))
R> compPlot(M2, index = 8, col= "forest green",
+             add = FALSE, just.theta = TRUE)
R> abline(h = 0, col = "orange", lty = 3, lwd = 2)
R> title(expression(zeta[t]))
R> plot(y.fit, type = "n", ylim = c(-0.3, 0),
+       xlab = "time", ylab = "component CrIs")
R> compPlot(M2, index = 9, col = "steelblue",
+             add = TRUE, just.theta = TRUE)
R> abline(h = 0, col = "orange", lty = 3, lwd = 2)
R> title(expression(psi[list(NOI, t)]))
R> plot(y.fit, type = "n", ylim = c(0, 0.3),
+       xlab = "time", ylab = "component CrIs")
R> compPlot(M2, index = 10, col = "darkorange",
+             add = TRUE, just.theta = TRUE)
R> abline(h = 0, col = "orange", lty = 3, lwd = 2)
R> title(expression(psi[list(AMO, t)]))
```

The fitted object also reports `median.kt`, the median number of time steps until the transfer effect on the 0.15 quantile is less than or equal to 1e−3, as discussed in Section 2.4. In this example, the value is approximately 7.14, suggesting that the transfer effect persists for about seven months.

```
R> M2$median.kt
```

```
[1] 7.13966
```

We next hold out the final 18 months of the aligned `BTflow/climateIndices` sample to illustrate the forecasting workflow. Figure 8 overlays the filtered estimates before the forecast origin and the 18-step-ahead forecasts after it.

```
R> exdqlmForecast(start.t = length(flow.fit)-18, k = 18, m1 = M1)
R> exdqlmForecast(start.t = length(flow.fit)-18, k = 18, m1 = M2,
+                   add = TRUE, cols = c("forest green","green"))
```

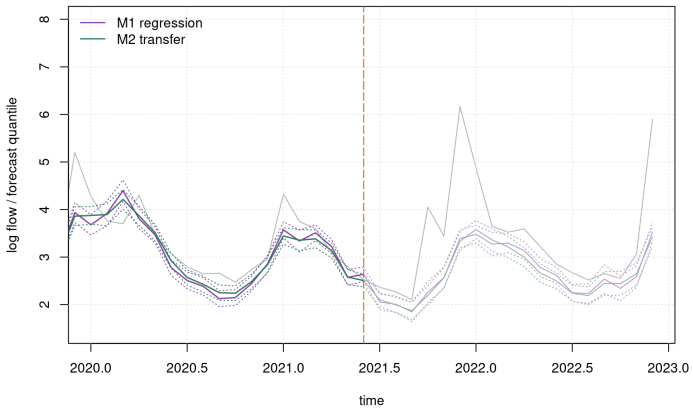

Figure 8: Example 3: Big Tree water flow. 18-step-ahead quantile forecast over the final holdout portion of the aligned monthly sample, ending in December 2022, overlaid on the log Big Tree water flow time series. The vertical dot-dashed line marks the forecast origin. Solid lines indicate mean estimates and dotted lines 95% CrI, with the filtered estimates shown before the forecast origin and the forecast estimates beyond it. Estimates from M_1, the quantile regression model, are seen in purple/pink. Estimates from M_2, the transfer function model, are seen in green.

Model	KL	CRPS	PPLC
`M1` regression	0.161	**0.350**	152.0
`M2` transfer function	**0.144**	0.353	**145.9**

Table 8: Example 3: Big Tree water flow. Diagnostic comparison between the dynamic regression model (`M1`) and the transfer-function model (`M2`). Lower values are preferred for all three criteria. Here CRPS assesses the full predictive distribution, PPLC assesses target-quantile posterior predictive loss, and KL summarizes one-step-ahead calibration discrepancy. Because the columns are the comparison metrics, bold entries indicate the smaller value within each column; displayed ties are left unbolded.

Finally, we use `exdqlmDiagnostics()` to compute the KL divergence, CRPS, and PPLC discussed in Section 3. Table 8 summarizes the resulting diagnostic comparison.

```
R> checks.out = exdqlmDiagnostics(M1, M2, ref = ref.samp,
+                                 cols = c("purple","forest green"),
+                                 plot = FALSE)
```

The diagnostics give a mixed but useful comparison. The transfer-function model has the smaller KL divergence and PPLC, whereas the direct regression model has the slightly smaller CRPS. The CRPS values are nearly identical, so we do not interpret the table as selecting a single clearly dominant model. Instead, the example illustrates how the package diagnostics expose different aspects of fit: the transfer-function model is favored by the calibration-oriented and target-quantile loss criteria, while the direct regression is slightly favored by the full-distribution CRPS criterion.

4.4. Static exAL regression on a simulated heteroskedastic example

To illustrate the static routines under sparse shrinkage, we consider a correlated Gaussian regression benchmark for which the target conditional quantile is known exactly. Let

$$\mathbf{x}_i \sim N_8(\mathbf{0}, \Sigma_x), \qquad (\Sigma_x)_{jk} = 0.5^{|j-k|},$$

with sparse signal

$$\boldsymbol{\beta}^\star = (3,\ 1.5,\ 0,\ 0,\ 2,\ 0,\ 0,\ 0)^\top.$$

We use the following benchmark data-generating mechanism. For each target quantile level p_0, we generate

$$Y_i^{(p_0)} = \mathbf{x}_i^\top \boldsymbol{\beta}^\star + \sigma_\varepsilon\{Z_i - \Phi^{-1}(p_0)\}, \qquad Z_i \sim N(0,1), \qquad \sigma_\varepsilon = 1.5.$$

Then

$$Q_Y(p_0 \mid \mathbf{x}_i) = \mathbf{x}_i^\top \boldsymbol{\beta}^\star,$$

so the true p_0-quantile is the same sparse linear signal at each fitted quantile level. We use $n = 160$ training observations and an independent holdout sample of size 800, and fit separate static exAL models at $p_0 \in \{0.05, 0.25, 0.50\}$. Both fits use the regularized horseshoe (RHS) prior of Nishimura and Suchard (2023), implemented in the package through `beta_prior = "rhs_ns"`, with $\tau_0 = 0.15$, fixed slab $\zeta^2 = 9$, and an unshrunk intercept. We fit a VB approximation with `exalStaticLDVB()` using `n.samp = 3000`, and an MCMC fit with `exalStaticMCMC()` using `n.burn = 2000` and `n.mcmc = 3000`; the MCMC routine uses its default `slice` kernel and is warm-started from the VB fit. The lower-tail VB fit at $p_0 = 0.05$ converges more slowly, so we allow a larger VB iteration budget there. Setting `al.ind = TRUE` (or equivalently `dqlm.ind = TRUE`) would recover the AL special case discussed in Section 2, but here we focus on the general static exAL model under sparse RHS shrinkage.

```
R> Sigma.x = 0.5 ^ as.matrix(dist(1:8))
R> beta.true = c(3, 1.5, 0, 0, 2, 0, 0, 0)
R> rhs.ctrl = list(tau0 = 0.15, a_zeta = 2, b_zeta = 9,
                   zeta2_fixed = 9, shrink_intercept = FALSE)
R> set.seed(20260718)
R> sim.y = function(X.raw, p0) {
      drop(X.raw %*% beta.true) + 1.5 * (rnorm(nrow(X.raw)) - qnorm(p0))
    }
R> X.raw = MASS::mvrnorm(160, mu = rep(0, 8), Sigma = Sigma.x)
R> X = cbind(1, X.raw)
R> p.grid = c(0.05, 0.25, 0.50)
R> y.train = lapply(p.grid, function(p0) sim.y(X.raw, p0))
R> M.ldvb = lapply(p.grid, function(p0) exalStaticLDVB(
+                    y = y.train[[which(p.grid == p0)]], X = X, p0 = p0,
+                    beta_prior = "rhs_ns",
+                    beta_prior_controls = rhs.ctrl,
+                    max_iter = ifelse(p0 == 0.05, 420, 260),
+                    n.samp = 3000,
+                    tol = 1e-4, verbose = FALSE))
R> M.mcmc = lapply(p.grid, function(p0) exalStaticMCMC(
+                    y = y.train[[which(p.grid == p0)]], X = X, p0 = p0,
+                    beta_prior = "rhs_ns",
+                    beta_prior_controls = rhs.ctrl,
```

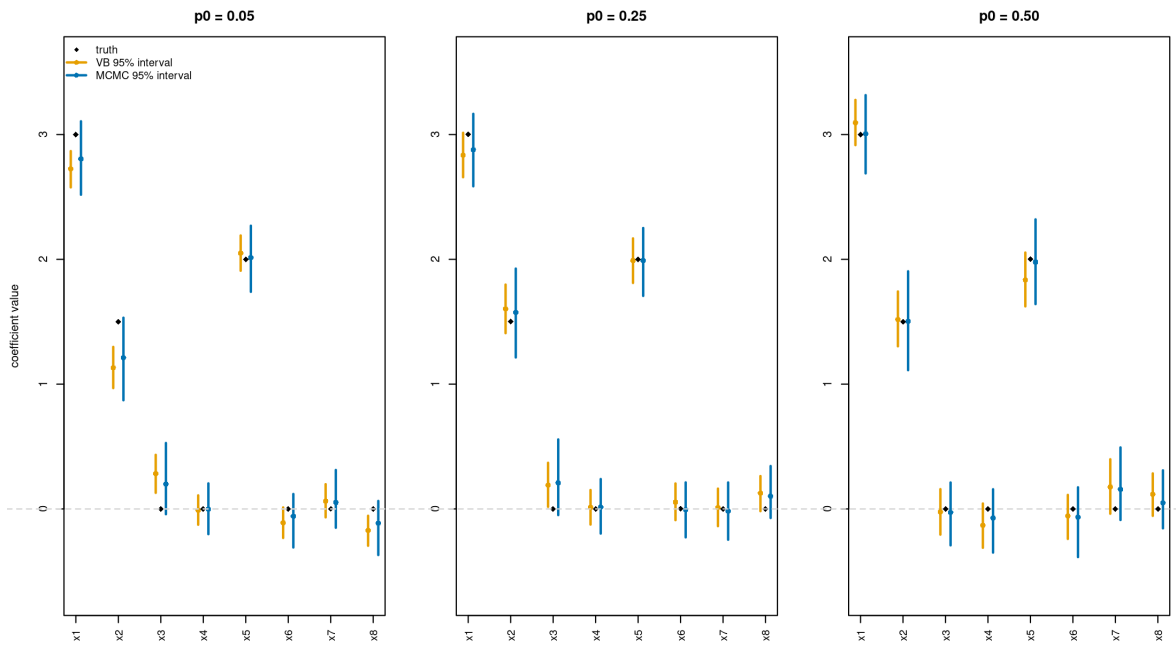

Figure 9: Example 4: sparse static exAL regression under the correlated Gaussian benchmark with the RHS prior. Black diamonds denote the true non-intercept coefficients, orange intervals denote VB 95% posterior intervals, and blue intervals denote MCMC 95% posterior intervals. Panels correspond to p0 = 0.05, 0.25, and 0.50.

```
+                   n.burn = 2000, n.mcmc = 3000,
+                   init.from.vb = TRUE,
+                   verbose = FALSE))
```

For static fits, the helper function `exalStaticDiagnostics()` summarizes fitted quantiles on a common design matrix and, when holdout responses or a reference quantile are supplied, reports predictive evaluation metrics such as check loss and reference-quantile error. In this simulated example, the true coefficient vector is also known, so we separately compute the active-signal RMSE and null-coefficient MAE used in Table 9. For real data applications, these coefficient-recovery summaries would be replaced by posterior intervals and holdout predictive diagnostics. For example, on an independent holdout design:

```
R> X.hold.raw = MASS::mvrnorm(800, mu = rep(0, 8), Sigma = Sigma.x)
R> X.hold = cbind(1, X.hold.raw)
R> ref.hold = drop(X.hold %*% c(0, beta.true))
R> y.hold = sim.y(X.hold.raw, 0.25)
R> exalStaticDiagnostics(M.ldvb[[2]], M.mcmc[[2]],
+                   X = X.hold, y = y.hold, ref = ref.hold,
+                   plot = FALSE)
```

Figure 9 shows that both methods recover the active coefficients and shrink the null coefficients toward zero across the lower tail, lower-middle quantile, and median. The lower-tail fit at $p_0 = 0.05$ remains the most demanding case, but both methods still recover the sparse signal

p_0	method	runtime (s)	active RMSE	null MAE	holdout qRMSE
0.05	VB	**22.50**	0.267	0.128	0.714
	MCMC	156.91	**0.201**	**0.085**	**0.515**
0.25	VB	**13.33**	0.112	0.079	0.302
	MCMC	149.08	**0.082**	**0.070**	**0.267**
0.50	VB	**2.51**	0.110	0.101	0.382
	MCMC	155.25	**0.013**	**0.074**	**0.367**

Table 9: Example 4: runtime and sparse-signal recovery metrics for the static exAL fits under the RHS prior. The VB rows use `n.samp = 3000`, and the MCMC rows use 2000 burn-in iterations and 3000 retained draws. The active RMSE is computed over the three nonzero coefficients, the null MAE is computed over the five zero coefficients, and the holdout quantile RMSE is computed against the true target quantile on an independent holdout sample of size 800. Because the columns are the comparison metrics, bold entries indicate the smaller value within each quantile block; displayed ties are left unbolded.

structure. The median panel is especially clean, with the MCMC reference intervals covering all plotted coefficients, while the VB fit remains visually close for the active signals.

Table 9 reinforces the visual comparison. The MCMC reference fit is better on the active-signal RMSE, the null-coefficient MAE, and the holdout quantile RMSE at all three quantile levels. The advantage is mild at $p_0 = 0.05$ and $p_0 = 0.25$, but it is especially pronounced for the active coefficients at $p_0 = 0.50$, where the MCMC active-signal RMSE is much smaller. The VB fit is nevertheless dramatically faster in all three cases, with runtime reductions of roughly 7-fold at $p_0 = 0.05$, 11-fold at $p_0 = 0.25$, and 62-fold at $p_0 = 0.50$. In this benchmark, the VB approximation remains useful as a rapid first-pass screen, but the MCMC fit provides the cleaner coefficient-recovery and uncertainty illustration. Accordingly, the static RHS specification still supports both roles cleanly: VB for fast screening and model exploration, and static MCMC as the reference method when higher-fidelity posterior uncertainty quantification is required.

5. Conclusion

In this paper we presented the R package **exdqlm** for Bayesian estimation and analysis of flexible dynamic quantile linear models. The package provides posterior simulation via MCMC and fast approximate posterior inference via VB for dynamic state-space quantile models, while also supporting transfer-function exDQLMs, static exAL quantile regression with shrinkage priors, forecasting, model diagnostics, and post hoc posterior-predictive synthesis across separately fitted quantiles. Together, these functions provide a practical framework for dynamic and static quantile modeling beyond the AL special case while preserving a computationally convenient latent-variable representation.

The examples illustrate the main workflows supported by the package. The dynamic examples show how **exdqlm** can be used to construct state-space models, estimate dynamic quantiles, compare the exDQLM with its AL special case, examine transfer function effects, and evaluate model fit through diagnostics and forecasting. The static example shows how the

same framework can be used for sparse quantile regression with both VB and MCMC, with VB serving as a fast first-pass approximation and MCMC providing the reference analysis when more faithful posterior uncertainty quantification is required. Viewed together, these examples highlight the two software contributions that most clearly distinguish the package within the current ecosystem: Bayesian dynamic state-space quantile modeling and a unified workflow that extends from static regularized exAL regression to post hoc predictive synthesis.

References

Barata R, Prado R, Sansó B (2022). "Fast inference for time-varying quantiles via flexible dynamic models with application to the characterization of atmospheric rivers." *The Annals of Applied Statistics*, **16**(1), 247–271. doi:10.1214/21-AOAS1497.

Barlow RE, Bartholomew DJ, Bremner JM, Brunk HD (1972). *Statistical Inference Under Order Restrictions: The Theory and Application of Isotonic Regression*. John Wiley & Sons, New York.

Beal MJ (2003). *Variational algorithms for approximate Bayesian inference*. Ph.D. thesis, UCL (University College London).

Benoit D, Al-Hamzawi R, Yu K, Van den Poel D (2023). ***bayesQR**: Bayesian Quantile Regression*. R package version 2.4, URL https://CRAN.R-project.org/package=bayesQR.

Bürkner PC (2026). ***brms**: Special Family Functions for **brms** Models*. Package documentation, accessed 2026-04-19, URL https://paulbuerkner.com/brms/reference/brmsfamily.html.

Carter CK, Kohn R (1994). "On Gibbs sampling for state space models." *Biometrika*, **81**(3), 541–553.

Chernozhukov V, Fernández-Val I, Galichon A (2010). "Quantile and Probability Curves Without Crossing." *Econometrica*, **78**(3), 1093–1125. doi:10.3982/ECTA7880.

Fan K, Wu C, Ren J, Li X, Zhou F (2026). ***pqrBayes**: Bayesian Penalized Quantile Regression*. R package version 1.2.1, URL https://CRAN.R-project.org/package=pqrBayes.

Fasiolo M, Griffiths B (2025). ***qgam**: Smooth Additive Quantile Regression Models*. R package version 2.0.0, URL https://CRAN.R-project.org/package=qgam.

Fogarty B (2020). "**QuantReg.jl**: Quickstart Guide." Documentation page, accessed 2026-04-19, URL https://fogarty-ben.github.io/QuantReg.jl/v0.1/quickstart/.

Fox K (2012). "Solar Minimum; Solar Maximum." URL https://www.nasa.gov/mission_pages/sunearth/news/solarmin-max.html.

Frühwirth-Schnatter S (1994). "Data augmentation and dynamic linear models." *Journal of time series analysis*, **15**(2), 183–202.

Frumento P (2025). ***qrcm**: Quantile Regression Coefficients Modeling*. R package version 3.2, URL https://CRAN.R-project.org/package=qrcm.

Galarza CE, Benites L, Bourguignon M, Lachos VH (2024). ***lqr**: Robust Linear Quantile Regression*. R package version 5.2, URL https://CRAN.R-project.org/package=lqr.

Gelfand AE, Ghosh SK (1998). "Model choice: a minimum posterior predictive loss approach." *Biometrika*, **85**(1), 1–11.

Gneiting T, Raftery AE (2007). "Strictly Proper Scoring Rules, Prediction, and Estimation." *Journal of the American Statistical Association*, **102**(477), 359–378. doi:10.1198/016214506000001437.

Gonçalves K, Migon HS, Bastos LS (2017). "Dynamic quantile linear model: a Bayesian approach." *arXiv preprint arXiv:1711.00162*.

He X, Pan X, Tan KM, Zhou WX (2023). ***conquer**: Convolution-Type Smoothed Quantile Regression*. R package version 1.3.3, URL https://CRAN.R-project.org/package=conquer.

Huerta G, Jiang W, Tanner MA (2003). "Time series modeling via hierarchical mixtures." *Statistica Sinica*, pp. 1097–1118.

Koenker R (2005). *Quantile regression*. Cambridge University Press, New York.

Koenker R (2025). ***quantreg**: Quantile Regression*. R package version 6.1, URL https://CRAN.R-project.org/package=quantreg.

Kottas A, Krnjajić M (2009). "Bayesian semiparametric modelling in quantile regression." *Scandinavian Journal of Statistics*, **36**(2), 297–319.

Kozumi H, Kobayashi G (2011). "Gibbs sampling methods for Bayesian quantile regression." *Journal of statistical computation and simulation*, **81**(11), 1565–1578.

Kullback S, Leibler RA (1951). "On information and sufficiency." *The annals of mathematical statistics*, **22**(1), 79–86.

MathWorks (2026). "Working with Quantile Regression Models." Documentation page, accessed 2026-04-19, URL https://www.mathworks.com/help/stats/working-with-quantile-regression-models.html.

Muggeo VMR (2025). ***quantregGrowth**: Non-Crossing Additive Regression Quantiles and Non-Parametric Growth Charts*. R package version 1.7-2, URL https://CRAN.R-project.org/package=quantregGrowth.

Nishimura A, Suchard MA (2023). "Shrinkage with shrunken shoulders: Gibbs sampling shrinkage model posteriors with guaranteed convergence rates." *Bayesian Analysis*, **18**(2), 367–390. doi:10.1214/22-BA1308.

NOAA Physical Sciences Laboratory (2026). "Climate Indices: Monthly Atmospheric and Ocean Time Series." URL https://psl.noaa.gov/data/climateindices/.

Ostwald D, Kirilina E, Starke L, Blankenburg F (2014). "A tutorial on variational Bayes for latent linear stochastic time-series models." *Journal of Mathematical Psychology*, **60**, 1–19.

Ou L, Hunter M, Chow SM, Ji L, Chen M, Hung HJ, Lee J, Li Y, Park J (2021). ***dynr***: *Dynamic Models with Regime-Switching*. R package version 0.1.16-2, URL https://CRAN.R-project.org/package=dynr.

Petris G, Gilks W (2018). ***dlm***: *Bayesian and Likelihood Analysis of Dynamic Linear Models*. R package version 1.1-5, URL https://CRAN.R-project.org/package=dlm.

Prado R, Molina F, Huerta G (2006). "Multivariate time series modeling and classification via hierarchical VAR mixtures." *Computational Statistics & Data Analysis*, **51**(3), 1445–1462.

Reich BJ, Bondell HD, Wang HJ (2009). "Flexible Bayesian quantile regression for independent and clustered data." *Biostatistics*, **11**(2), 337–352.

Rosenblatt M (1952). "Remarks on a multivariate transformation." *The annals of mathematical statistics*, **23**(3), 470–472.

Sherwood B, Li S, Maidman A (2026). ***rqPen***: *Penalized Quantile Regression*. R package version 4.2, URL https://CRAN.R-project.org/package=rqPen.

statsmodels developers (2025). "**statsmodels**: `QuantReg` Documentation." Version 0.14.6 documentation, accessed 2026-04-19, URL https://www.statsmodels.org/stable/generated/statsmodels.regression.quantile_regression.QuantReg.html.

Taddy MA, Kottas A (2010). "A Bayesian nonparametric approach to inference for quantile regression." *Journal of Business & Economic Statistics*, **28**(3), 357–369.

Tokdar S, Cunningham E (2025). ***qrjoint***: *Joint Estimation in Linear Quantile Regression*. R package version 2.0-11, URL https://CRAN.R-project.org/package=qrjoint.

US Geological Survey (2016). "National Water Information System data available on the World Wide Web (USGS Water Data for the Nation)." URL http://waterdata.usgs.gov/nwis/.

Wang C, Blei DM (2013). "Variational Inference in Non-Conjugate Models." *arXiv preprint arXiv:1209.4360*. URL https://arxiv.org/abs/1209.4360.

West M, Harrison PJ, Migon HS (1985). "Dynamic generalized linear models and Bayesian forecasting." *Journal of the American Statistical Association*, **80**(389), 73–83.

Yan Y, Zheng X, Kottas A (2025). "A New Family of Error Distributions for Bayesian Quantile Regression." *Bayesian Analysis*. doi:10.1214/25-BA1507.

Yu K, Moyeed RA (2001). "Bayesian quantile regression." *Statistics & Probability Letters*, **54**(4), 437–447.

A. Global backend options

Option	Purpose	Default
`exdqlm.use_cpp_kf`	Enables or disables the C++ Kalman-filter bridge in dynamic routines.	`TRUE`
`exdqlm.compute_elbo`	Enables or disables ELBO computation in variational routines.	`TRUE`
`exdqlm.tol_elbo`	Sets the ELBO stabilization tolerance when ELBO checks are enabled.	`1e-6`
`exdqlm.use_cpp_builders`	Enables or disables C++ builders for state-space model blocks.	`FALSE`
`exdqlm.use_cpp_samplers`	Enables or disables C++ latent-variable samplers in dynamic routines.	`FALSE`
`exdqlm.use_cpp_postpred`	Enables or disables the C++ posterior-predictive sampling path.	`FALSE`
`exdqlm.use_cpp_mcmc`	Enables or disables C++ backend routing in dynamic MCMC routines.	`TRUE`
`exdqlm.cpp_mcmc_mode`	Chooses dynamic MCMC routing mode (`"strict"` for parity path, `"fast"` for accelerated path).	`"fast"`
`exdqlm.cpp_threads`	Sets the thread cap for eligible OpenMP-enabled C++ paths.	`1L`

Table 10: Package-level runtime options exposed through `options(...)` for backend routing and ELBO monitoring in **exdqlm**. Function-level inference controls are documented separately in the function interfaces (for example, Table 3).

Affiliation:

Antonio Aguirre and Raquel Barata
Department of Statistics
University of California Santa Cruz
Santa Cruz, CA 95064
E-mail: jaguir26@ucsc.edu, raquel.a.barata@gmail.com

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd